UNIVERSIDADE CATÓLICA DE BRASÍLIA

Laboratório de Banco de Dados — GPE17M40083 — 2026/2

Relatório Final — Etapa 2

Operação, interface e desempenho — Sistema de Gestão de Eventos Acadêmicos

Item	Informação
Professor	Samuel Novais Moura Júnior
Turma / Grupo	Ciência da Computação — Grupo 10
Integrantes	Isaac Barbosa da Silva; Átila de Melo Cantoara; Victor Lemos de Abreu Oliveira; João Pedro Carneiro Ibiapino; Jayme Bernardino da Silva Neto
SGBD / Aplicação	MySQL 8.0 / Python 3.10+ com Flask
Entrega e apresentação	Entrega no AVA em 22/11/2026; apresentação em 23/11/2026 (aula 17)
Repositório	https://github.com/JaymeNeto2019/projeto-banco-eventos-academicos

> Como ler este relatório: a seção 0 reúne a Etapa 1 revisada; as seções 1 a 7 seguem os artefatos B1 a B7 do enunciado; a seção 8 traz a declaração de uso de IA. As evidências em imagem são capturas do MySQL Workbench feitas pelo grupo no próprio ambiente; as evidências em texto ficam em docs/evidencias/.

0. Etapa 1 revisada

O banco modela a gestão de eventos acadêmicos: instituições, salas, eventos (com histórico datado de status), atividades e sessões, pessoas e participantes, organizadores, inscrições em múltiplas atividades com controle de vagas, e certificados com carga horária. Os artefatos da Etapa 1 (A1 a A5) permanecem no repositório e compõem este relatório:

Artefato	Arquivo no repositório	Conteúdo
A1 — Escopo e regras	docs/relatorio-etapa1.pdf	Descrição do domínio e vinte regras de negócio (RN01–RN20)
A2 — MER conceitual	docs/mer-conceitual.pdf / .png / .drawio	Notação de Engenharia da Informação (pé de galinha)
A3 — Dicionário de dados	docs/dicionario-dados.pdf	Entidades e atributos com domínio e obrigatoriedade
A4 — Modelo lógico	docs/modelo-logico.pdf / .png	Esquema relacional e decisões de mapeamento
A5 — Normalização	docs/relatorio-etapa1.pdf	Dependências funcionais e verificação da 3FN
A6–A8 — SQL	sql/01_ddl.sql, 02_carga.sql, 03_consultas.sql	Script físico, carga e quinze consultas

0.1 Devolutiva da Etapa 1 e correções

A devolutiva da Etapa 1 foi tratada item a item na tabela abaixo. O redesenho do MER segue pendente.

Observação do professor	O que foi alterado	Onde (arquivo/seção)
RN15, RN22 e RN24 sem verificação (atendidas "por consulta" sem a consulta no repositório).	Criada uma consulta de verificação para cada uma (resultado vazio = regra respeitada).	sql/10_verificacao_regras.sql
RN06 (categoria não pode ser pai de si mesma): sem mecanismo no banco; atendimento deve constar como "gatilho", não "CHECK".	Criados os gatilhos BEFORE INSERT e BEFORE UPDATE em categoria_evento (SIGNAL SQLSTATE 45000). Coluna de atendimento da RN06 corrigida para "gatilho". Comentário "complemento do CHECK" corrigido para "complemento do gatilho". Ciclos A→B→A continuam detectados pela consulta de verificação.	sql/01_ddl.sql (final do arquivo); docs/relatorio-etapa1 (tabela de regras); sql/10_verificacao_regras.sql
Carga com nomes e títulos em série ("Semana de Tecnologia 2026 - Edição 01", "Registro 01", "Ana Duarte 01").	Carga reescrita: nomes e e-mails únicos e sem numeração; títulos de eventos e de atividades distintos e coerentes com a categoria (o título antigo do evento 4, por exemplo, não batia com a categoria); descrições específicas.	sql/02_carga.sql
Inscrições com datas incoerentes (inscrições em 2027 para eventos que terminam em 2026; inscrições em eventos ainda não abertos).	Toda inscrição passou a ter data entre a abertura do evento e o dia anterior ao início, e nunca posterior à data de referência da carga (30/10/2026). Eventos PLANEJADOS ficaram sem inscrições. O histórico de status de cada evento foi refeito em ordem cronológica coerente com essa data.	sql/02_carga.sql
Inscrições confirmadas em eventos cancelados.	Eventos CANCELADOS (38, 39 e 40) passam por PLANEJADO, ABERTO e CANCELADO e só têm inscrições CANCELADAS, sem presença.	sql/02_carga.sql

Observação do professor	O que foi alterado	Onde (arquivo/seção)
Faltam participantes sem inscrição e organizadores sem evento (o LEFT JOIN não mostra o caso sem correspondência).	Incluídos 6 participantes sem inscrição (ids 81 a 86) e 3 organizadores sem evento (ids 87 a 89, um deles inativo). Q07 e Q10 passam a mostrar linhas com contagem 0.	sql/02_carga.sql; consultas Q07 e Q10 em sql/03_consultas.sql
Dependência funcional de matricula_academica em PARTICIPANTE não consta na análise de normalização.	Acrescentada a seção 5.3: chave candidata, nula para quem não é aluno, relação em BCNF.	docs/relatorio-etapa1 (seção 5.3)
Resultado esperado das 15 consultas não consta no relatório.	Acrescentada a seção 0.5, com o número de linhas esperado e a leitura de cada consulta sobre a carga nova.	Este relatório, seção 0.5
MER: marcações PK/FK nos atributos, símbolo próprio da especialização, notação coerente com o texto (Engenharia da Informação) e reorganização das linhas em torno de PARTICIPANTE, CERTIFICADO e ORGANIZACAO_EVENTO.	PENDENTE: o desenho é refeito no draw.io pelo grupo.	docs/mer-conceitual.drawio

0.2 O que mudou no modelo físico desde a Etapa 1

Para a Etapa 2 foram acrescentados, sem remover nada do esquema original: as tabelas usuario_app (login da aplicação, com senha_hash) e auditoria (alimentada por gatilhos); quatro visões (vw_evento_status_atual, vw_relatorio_eventos, vw_agenda_publica, vw_validacao_certificado); quatro papéis MySQL (r_admin_eventos, r_organizador, r_consulta, r_autenticacao); e três índices de apoio aos estudos de desempenho. Também foram acrescentados à Etapa 1 os gatilhos trg_categoria_sem_autopai_ins e trg_categoria_sem_autopai_upd (RN06), no final de 01_ddl.sql.

0.3 Modelos (Etapa 1)

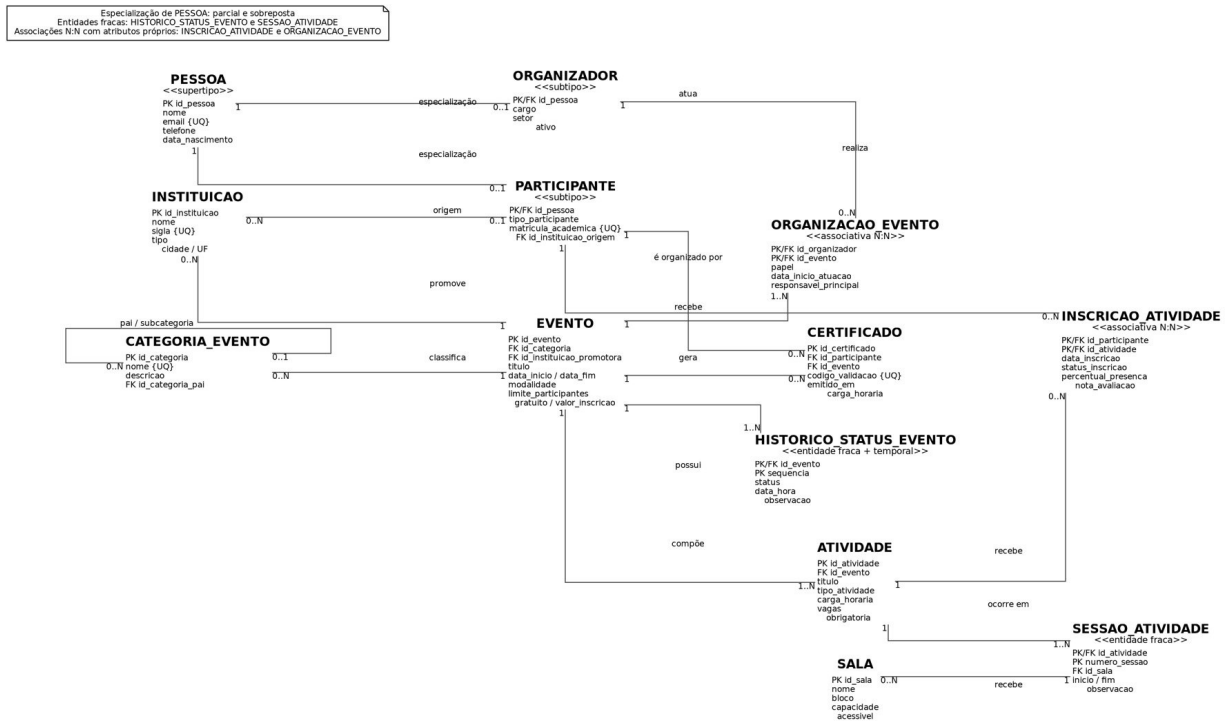


Figura 1 — MER conceitual (notação de Engenharia da Informação)

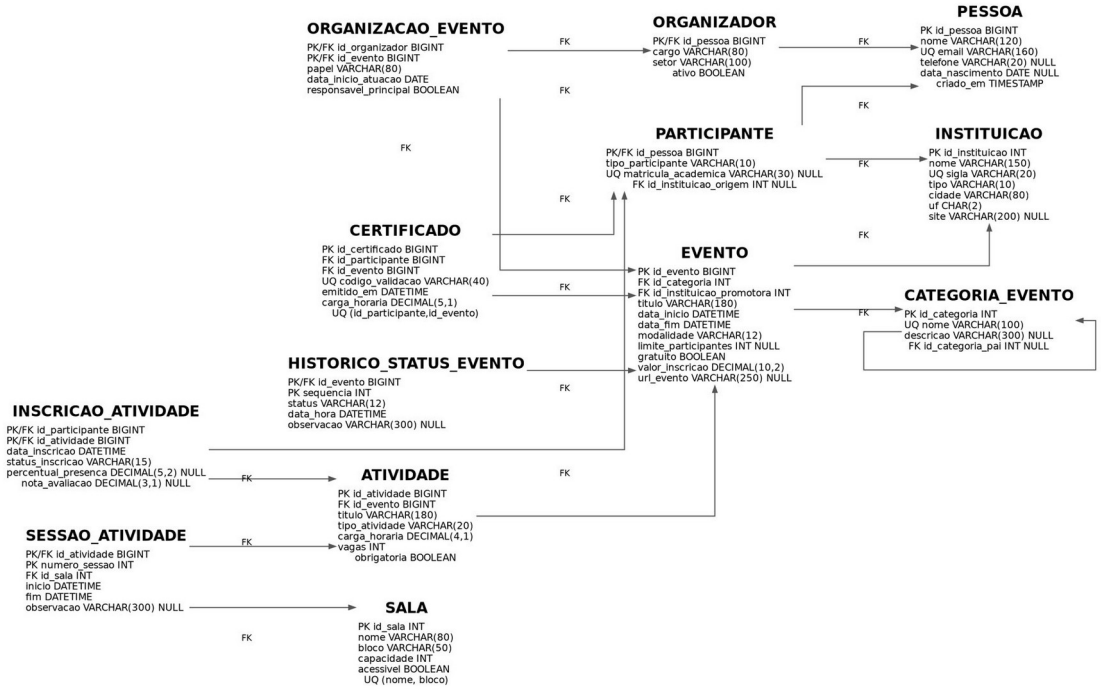


Figura 2 — Modelo lógico relacional

0.4 Verificação do banco no ambiente do grupo

Antes de iniciar a Etapa 2 o grupo executou a carga e as consultas no MySQL Workbench e conferiu os resultados.

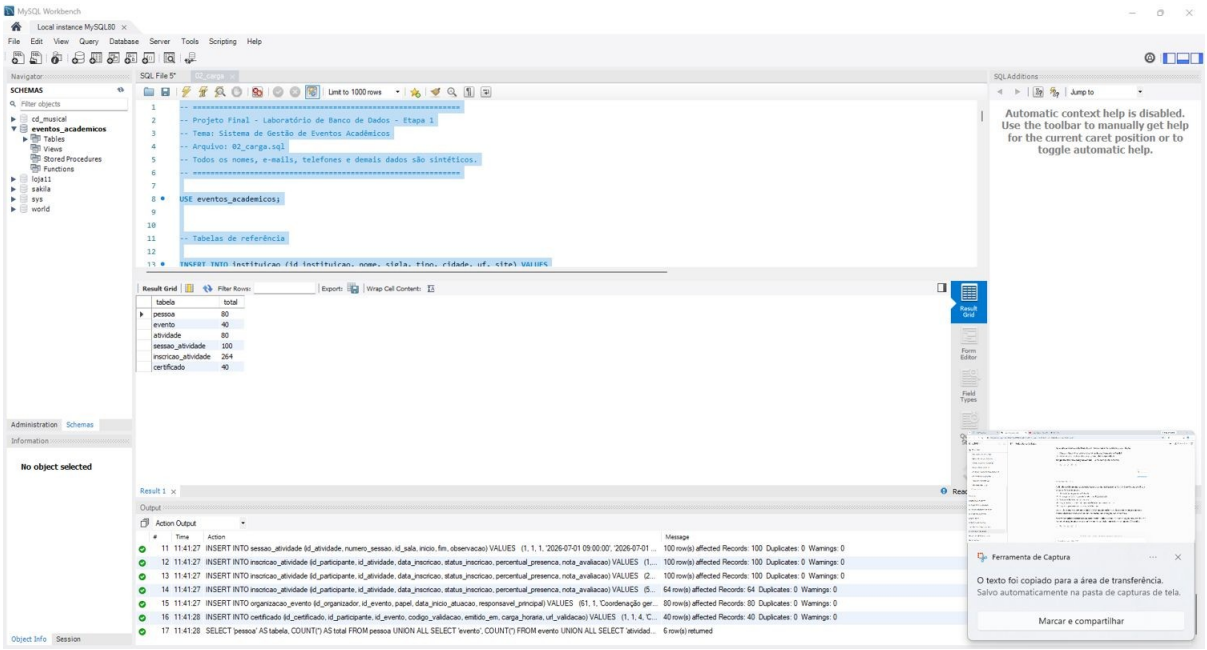


Figura 3 — Execução da carga de dados (02_carga.sql)

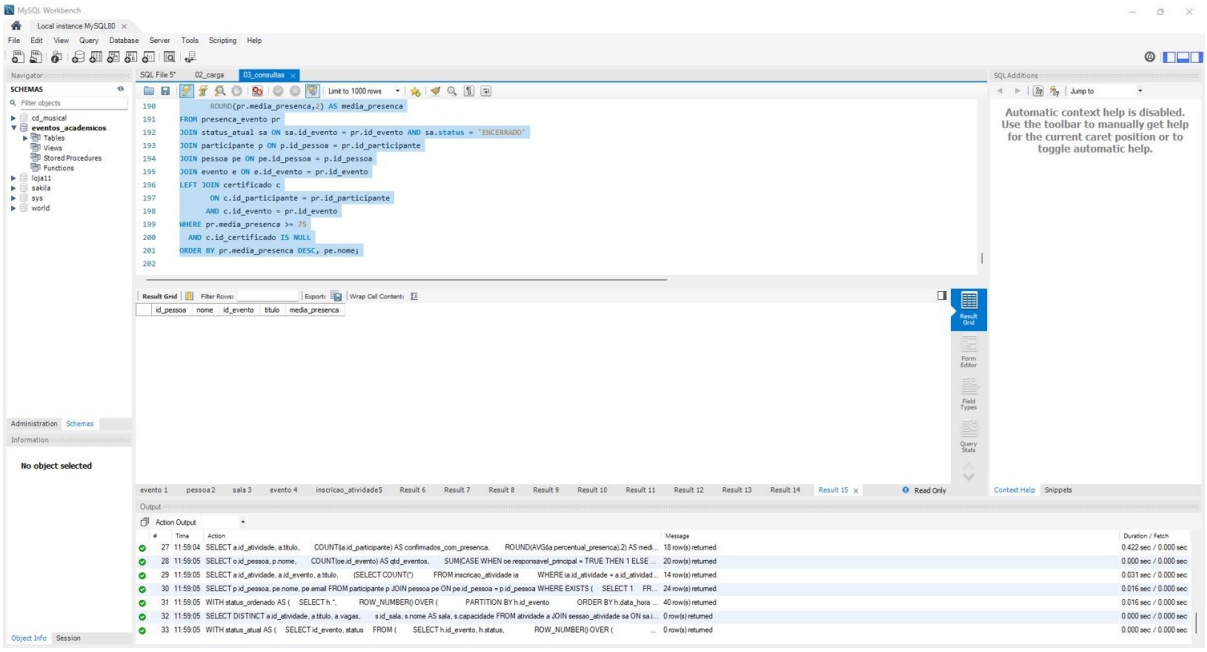


Figura 4 — Consultas da Etapa 1 (última consulta executada)

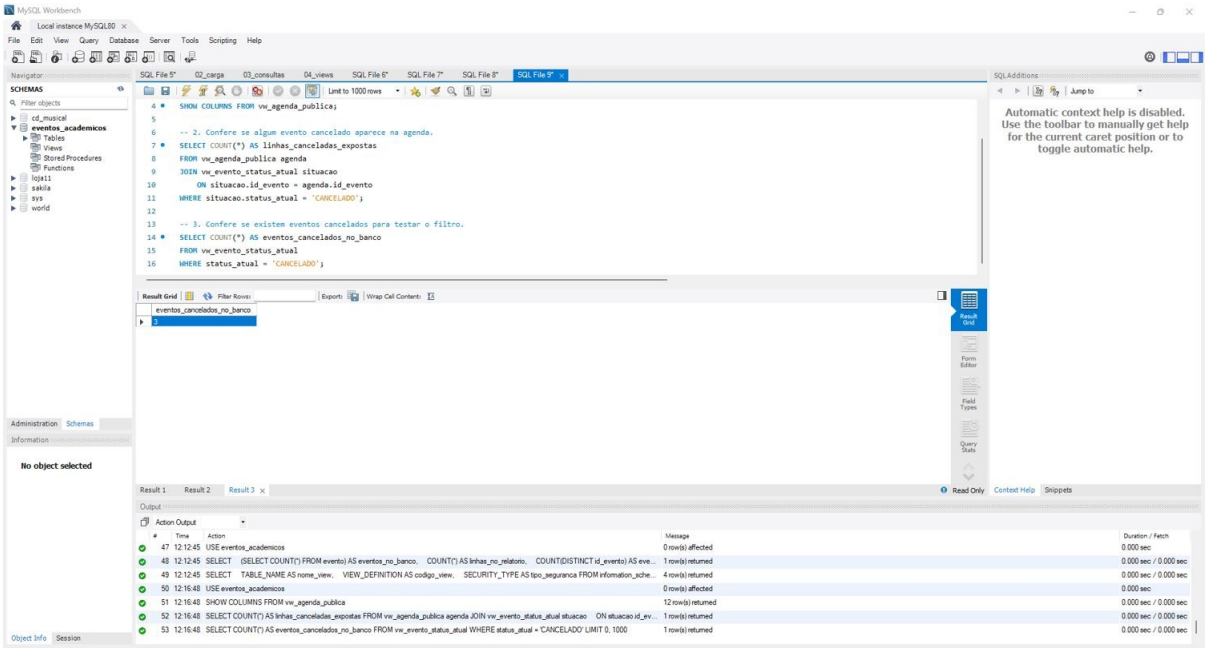


Figura 5 — Consulta dos três eventos cancelados no banco

0.5 Resultado esperado das 15 consultas (carga atual)

Número de linhas esperado para cada consulta de sql/03_consultas.sql sobre a carga reescrita. Q14 devolve vazio porque lista violações da RN18.

Consulta	Linhas	O que o resultado mostra
Q01	17	Eventos de 25/09 a 31/12/2026; o primeiro é o evento 11.
Q02	14	Pessoas com "Ana" em qualquer parte do nome (inclui Adriana, Mariana, Susana etc.).
Q03	5	Salas dos blocos A e B com 80 vagas ou mais; a maior é a Sala 105 (140).
Q04	8	Eventos sem URL pública cadastrada.
Q05	53	39 inscrições canceladas e 14 em lista de espera.
Q06	40	Todos os eventos têm duas atividades.
Q07	71	Participantes com contagem de inscrições; 6 aparecem com 0 (LEFT JOIN).

Consulta	Linhas	O que o resultado mostra
Q08	100	Uma linha por sessão de atividade, com sala e horário.
Q09	27	Atividades confirmadas com presença média de pelo menos 75%.
Q10	23	Organizadores com contagem de eventos; 3 aparecem com 0 (LEFT JOIN).
Q11	12	Atividades acima da média de inscrições do próprio evento.
Q12	21	Participantes inscritos em atividade de evento da categoria Computação.
Q13	40	Status atual: 15 ENCERRADO, 13 ABERTO, 9 PLANEJADO e 3 CANCELADO.
Q14	0	Nenhuma atividade viola a RN18 (vagas acima da capacidade da sala).
Q15	22	Participantes elegíveis a certificado (RN26) em eventos encerrados e sem certificado emitido.

Contagens obtidas simulando a mesma carga em SQLite; a ordem entre linhas empatadas pode variar no MySQL.

A COMPLETAR: executar 01_ddl.sql, 02_carga.sql e 03_consultas.sql numa base limpa e conferir se as contagens acima batem. Os prints das Figuras 3 a 5 foram tirados com a carga anterior e devem ser refeitos.

1. Aplicação com interface (B1)

A aplicação web em Python/Flask (pasta app/) está conectada ao banco da Etapa 1. A interface é simples de propósito: o que se avalia é o que ela revela sobre o banco.

Requisito B1	Como foi atendido	Onde
CRUD de 4+ entidades	Cinco CRUDs completos: instituição, sala, categoria de evento, evento e atividade (criar, listar, alterar, excluir)	/crud/<entidade>
2 consultas parametrizadas	Ocupação das atividades (JOIN + LEFT JOIN + GROUP BY + HAVING) e participação por instituição (LEFT JOIN + GROUP BY + HAVING + LIKE)	/consulta/ocupacao; /consulta/participacao
Relatório por visão	Uma linha por evento, lida da visão vw_relatorio_eventos	/relatorio
Operação multitabela em uma transação	Encerrar evento e emitir certificados: grava historico_status_evento e certificado em uma só transação	/operacoes/encerrar-evento
Erro de restrição compreensível	Mensagens traduzidas por nome de restrição (UNIQUE, CHECK, FK, permissão negada) em app/db.py; a exceção do driver nunca é exibida	Todas as telas

O arquivo app/teste_fumaca.py automatiza 33 verificações (login, CSRF, CRUD, erros amigáveis, consultas, injeção de SQL, transações e perfis); a saída registrada está em docs/evidencias/06_teste_fumaca.txt.

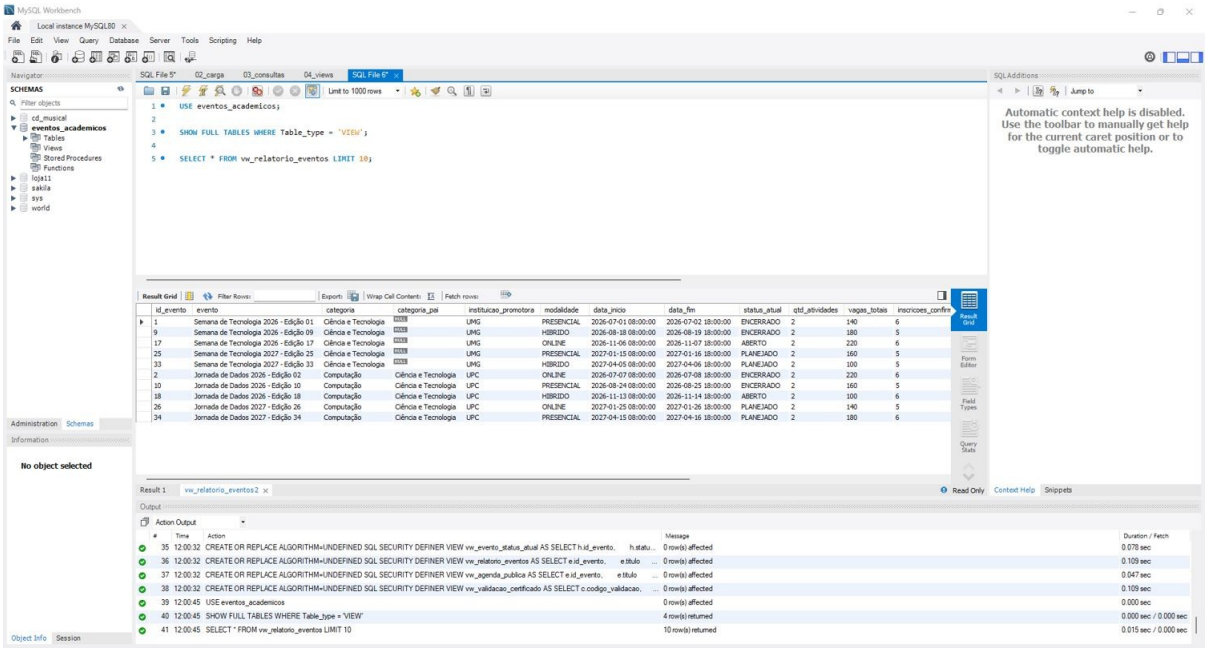


Figura 6 — Relatório consolidado alimentado pela visão vw_relatorio_eventos

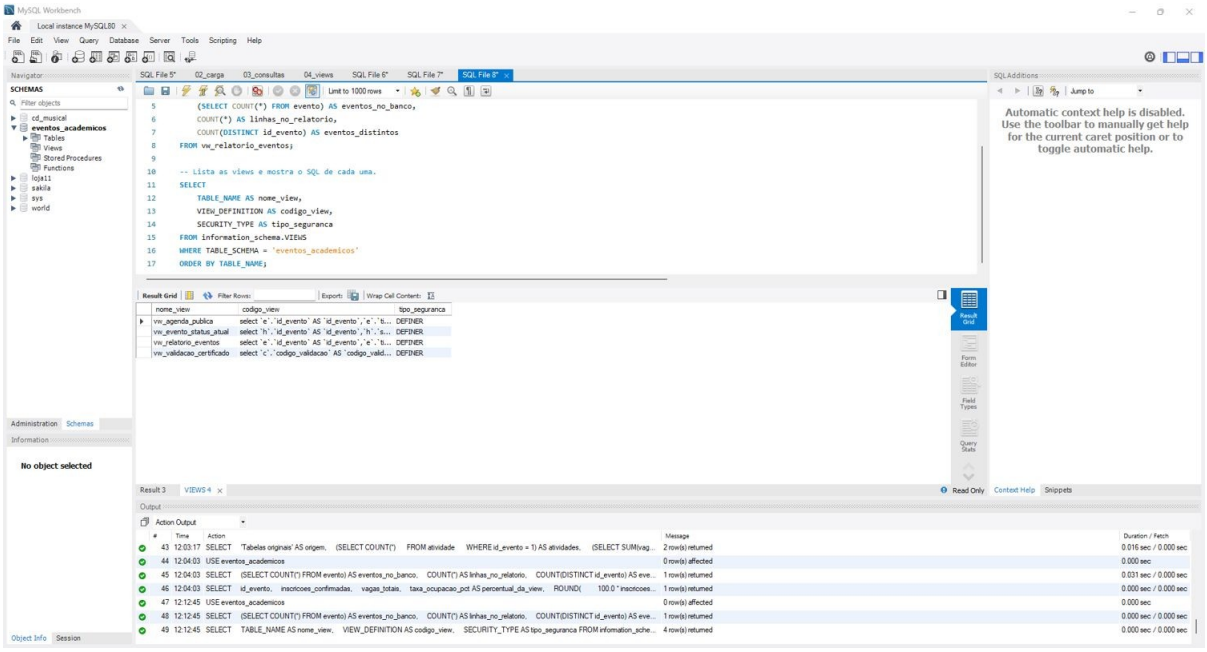


Figura 7 — Visões existentes no banco

A COMPLETAR: inserir prints da aplicação rodando: tela de login, lista e formulário de um CRUD, as duas consultas com filtros, a tela de relatório e uma mensagem de erro tratada (ex.: sala com capacidade 0 → "A capacidade da sala deve ser maior que zero.").

2. Camada de acesso a dados (B2)

Consultas parametrizadas. Todo valor vindo do usuário é enviado ao driver em uma tupla de parâmetros (%s); não há concatenação de valores em SQL. Nos CRUDs, o nome da tabela e das colunas vem do dicionário fixo app/entidades.py, e a rota rejeita (404) qualquer entidade fora dele.

Credenciais fora do código. Lidas de variáveis de ambiente/.env, que está no .gitignore; o repositório traz apenas .env.example. Há uma conta MySQL por perfil, escolhida em db.conectar() conforme o perfil do login.

2.1 Tentativa de injeção de SQL

Na tela “Participação por instituição” foram usadas as entradas abaixo. Em todas, a aplicação tratou o texto como dado e a tabela instituicao permaneceu intacta.

Tentativa	Entrada usada	Resultado
Busca normal	instituicao=UT	HTTP 200; 3 linhas
Tautologia	' OR '1'='1	HTTP 200; 0 linhas; nenhum erro exibido
UNION	' UNION SELECT email,1,1,1 FROM pessoa--	HTTP 200; 0 linhas; nenhum erro exibido
Comando destrutivo	ALUNO'; DROP TABLE instituicao;--	HTTP 200; 0 linhas; tabela continua existindo

Trecho do código (app.py): WHERE (%s IS NULL OR i.sigla LIKE CONCAT('%%', %s, '%%')) — os valores seguem em tupla ao driver. A saída completa está em docs/evidencias/05_sql_injection_e_concorrencia_app.txt.

3. Transações e controle de concorrência (B3)

3.1 Operação transacional documentada

A rota /operacoes/encerrar-evento executa, em transação explícita: START TRANSACTION → SELECT … FOR UPDATE no evento → valida o status atual → INSERT em historico_status_evento (status ENCERRADO) → INSERT … SELECT em certificado para quem tem presença mínima → COMMIT. Qualquer erro dispara ROLLBACK. A opção “Simular falha” provoca uma exceção depois dos INSERTs para demonstrar o desfazimento.

3.2 Sucesso, falha no meio e ROLLBACK

Em duas sessões do Workbench o grupo executou a transação sobre as duas tabelas: primeiro com COMMIT, depois com falha provocada por violação de chave (duplicidade) e conferência de que nada permaneceu.

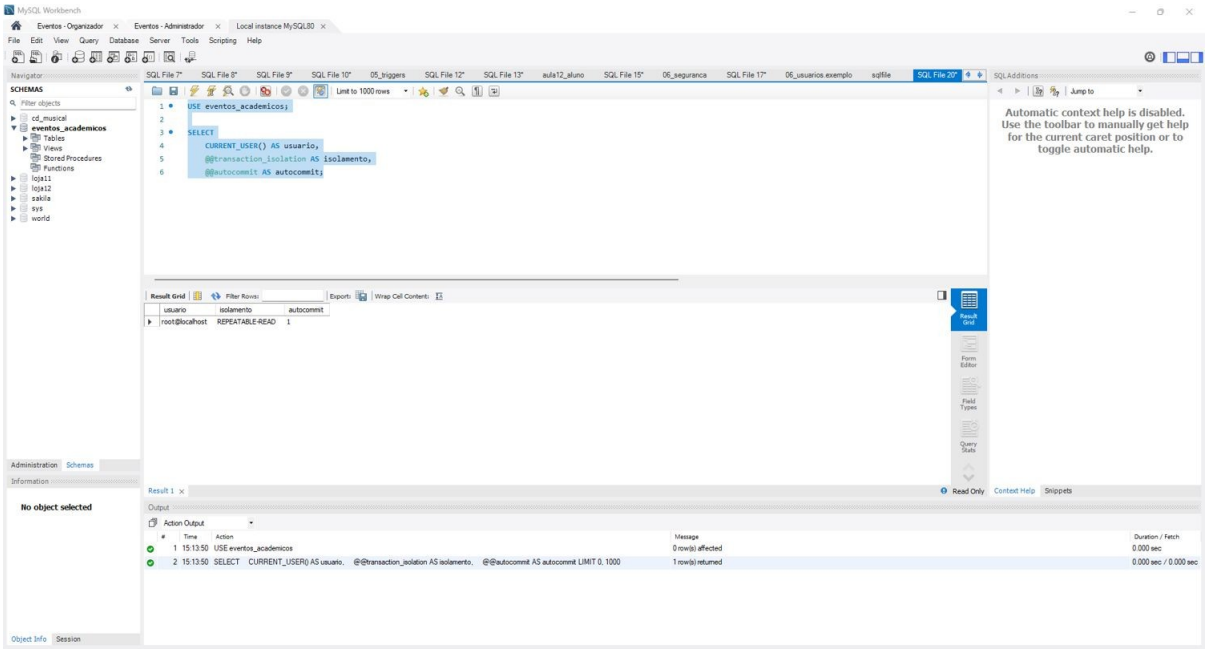


Figura 8 — Ambiente de transações: contagens iniciais das duas tabelas

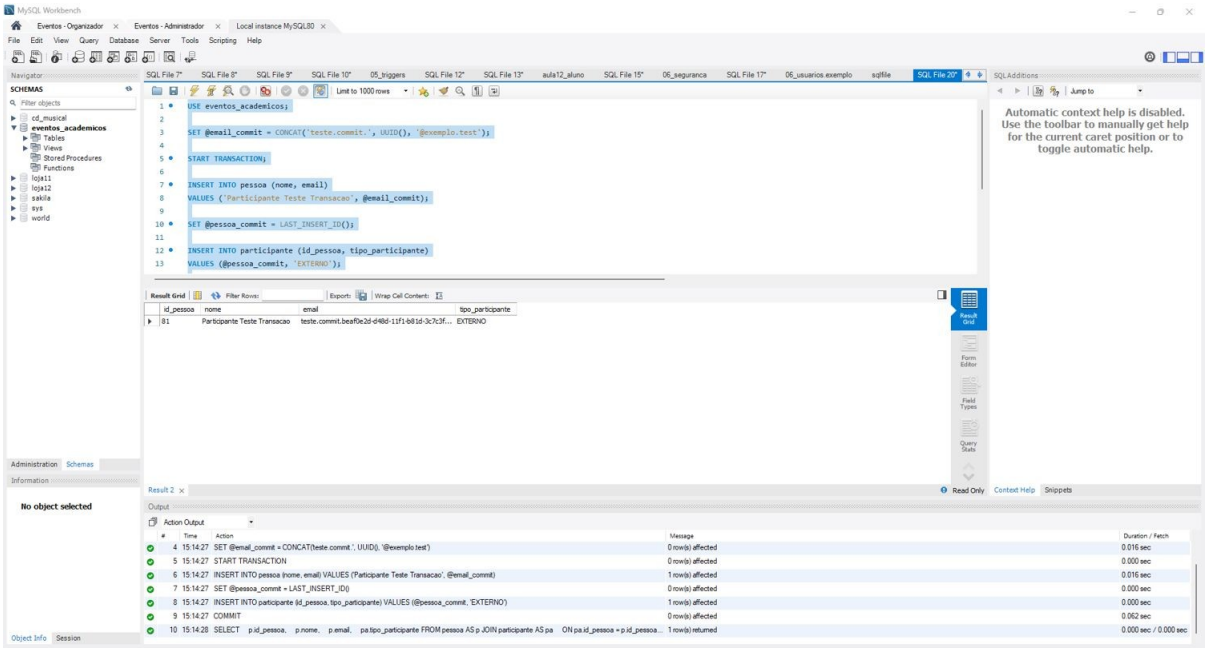


Figura 9 — Transação em duas tabelas concluída com COMMIT

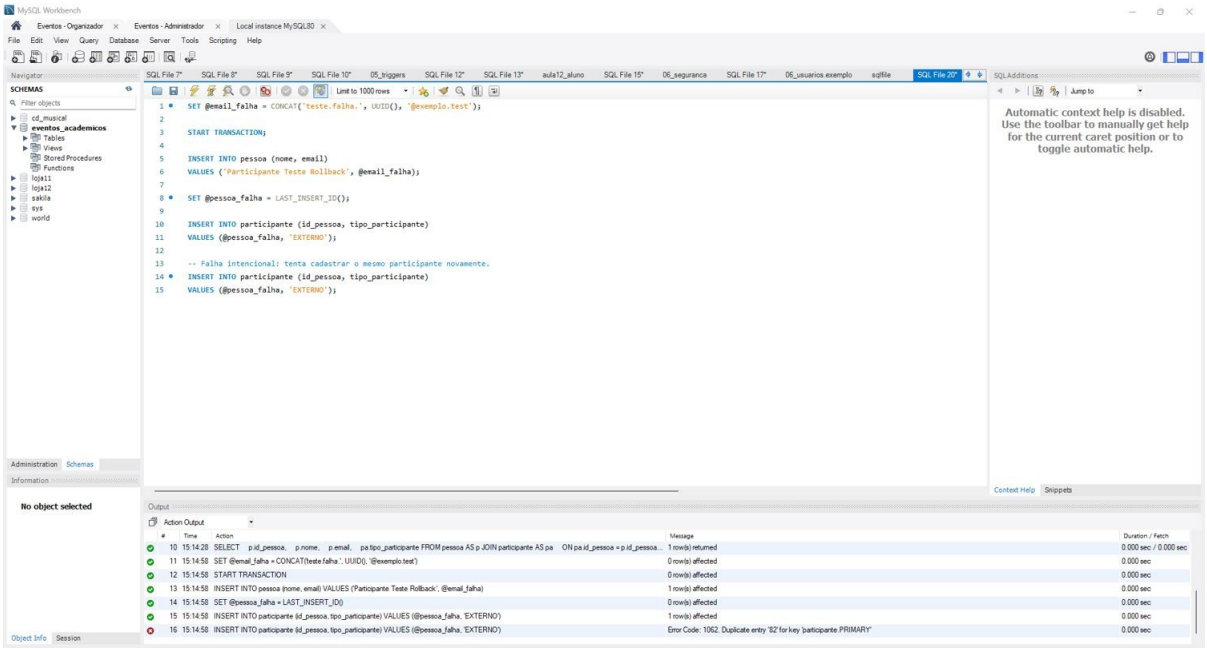


Figura 10 — Falha provocada no meio da transação (violação de restrição)

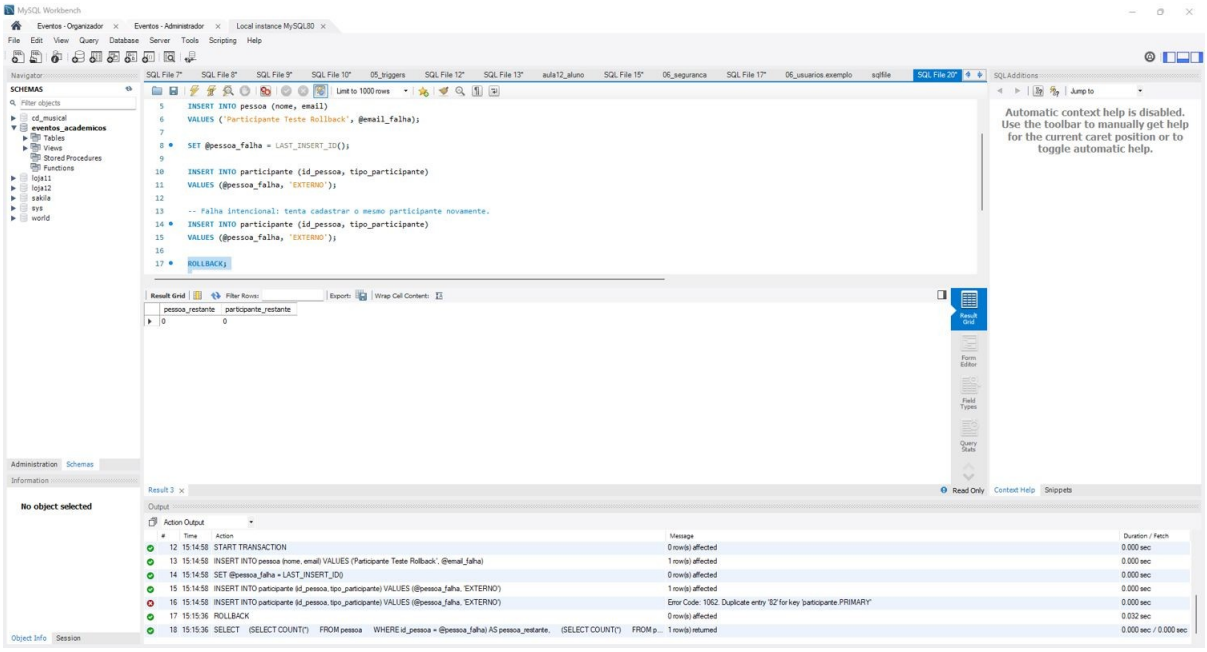


Figura 11 — ROLLBACK confirmado: as duas tabelas voltaram ao estado anterior

3.3 Experimento de concorrência com duas sessões

Nível de isolamento padrão do InnoDB: REPEATABLE READ. Foram usadas duas conexões simultâneas (Sessão A e Sessão B).

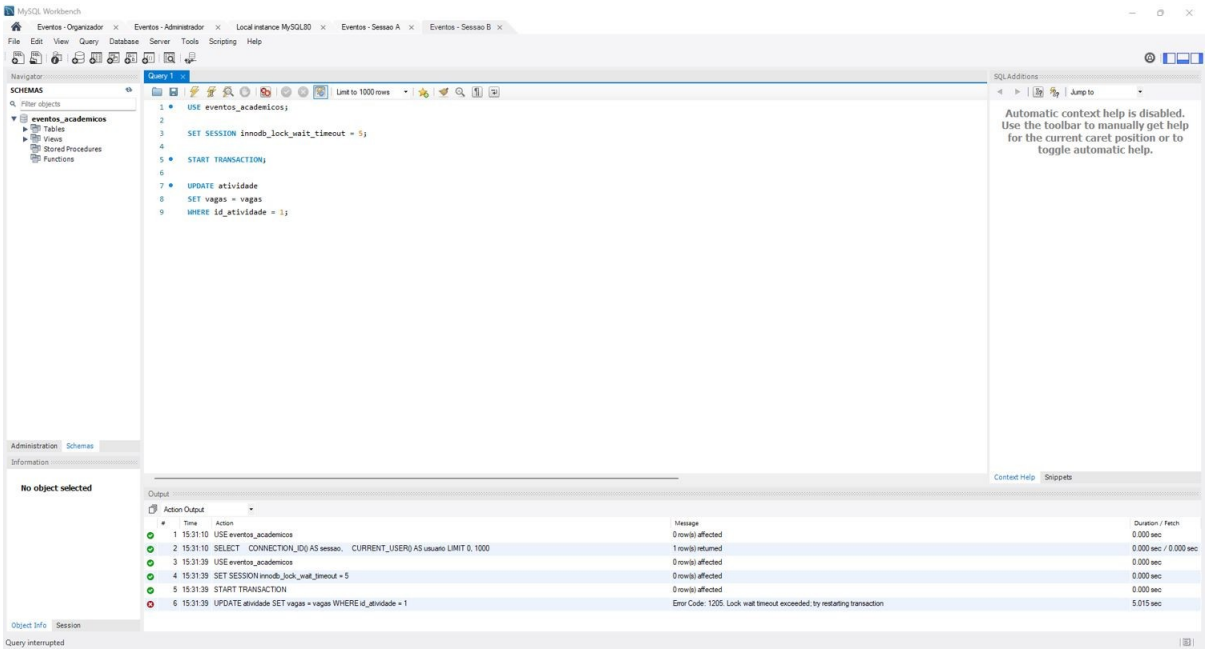


Figura 12 — Bloqueio entre sessões: a Sessão B espera a Sessão A liberar a linha

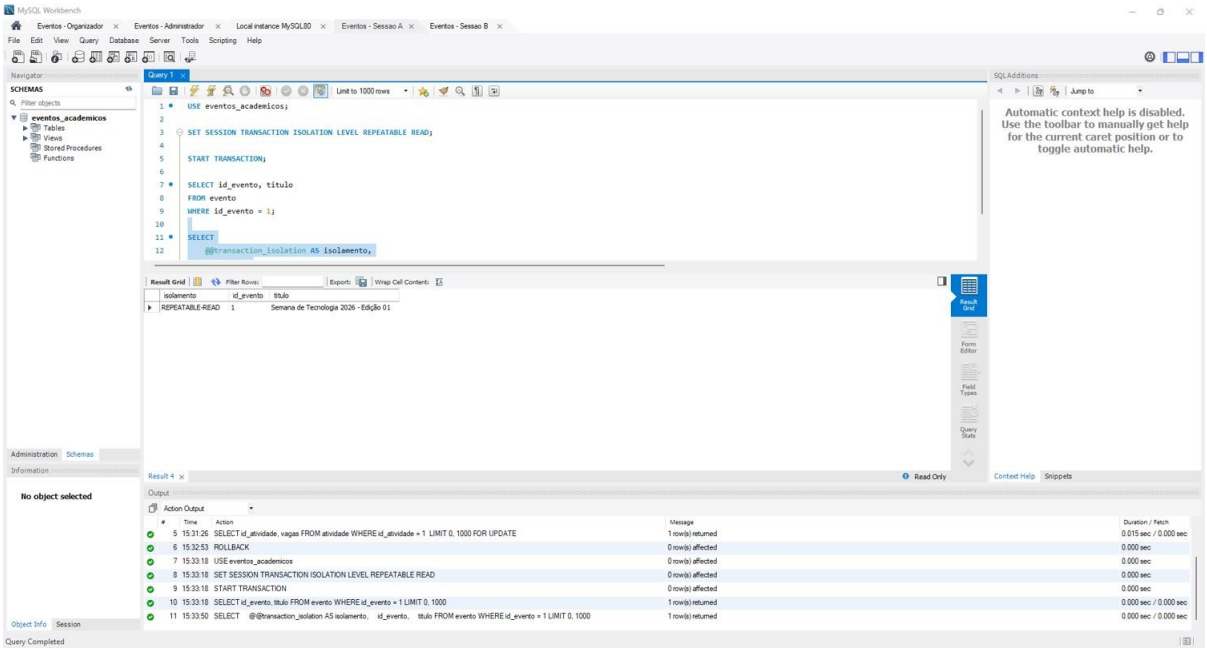


Figura 13 — REPEATABLE READ: a leitura dentro da transação se mantém mesmo após alteração confirmada por outra sessão

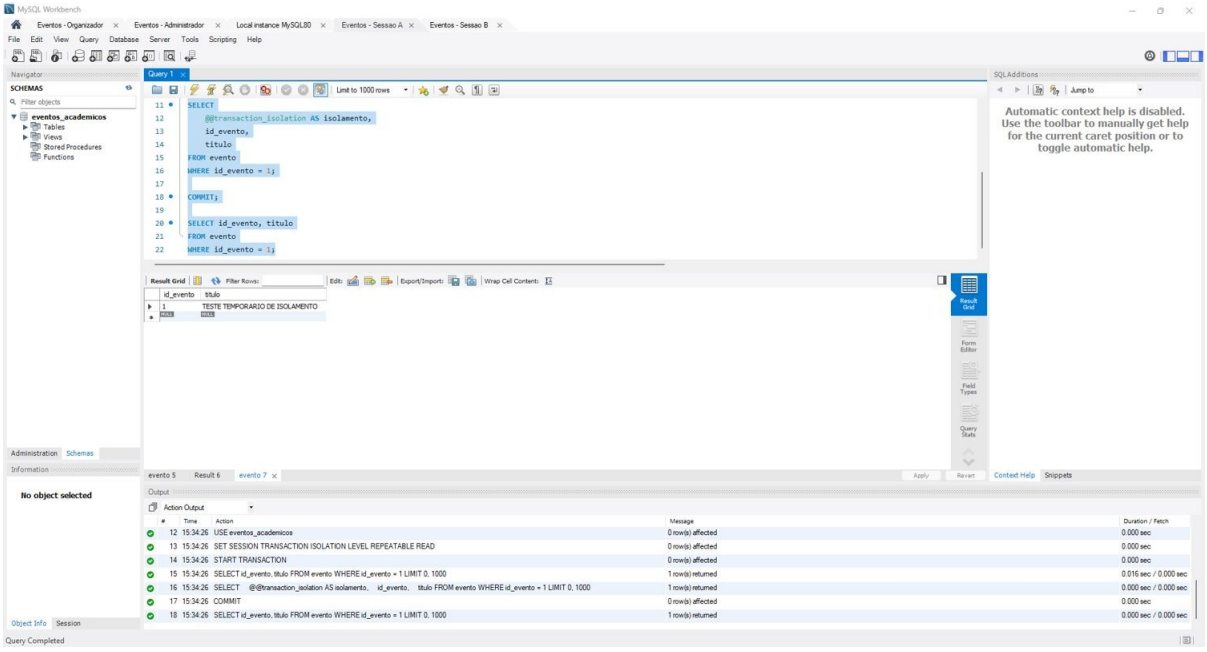


Figura 14 — Uma nova transação enxerga a alteração confirmada

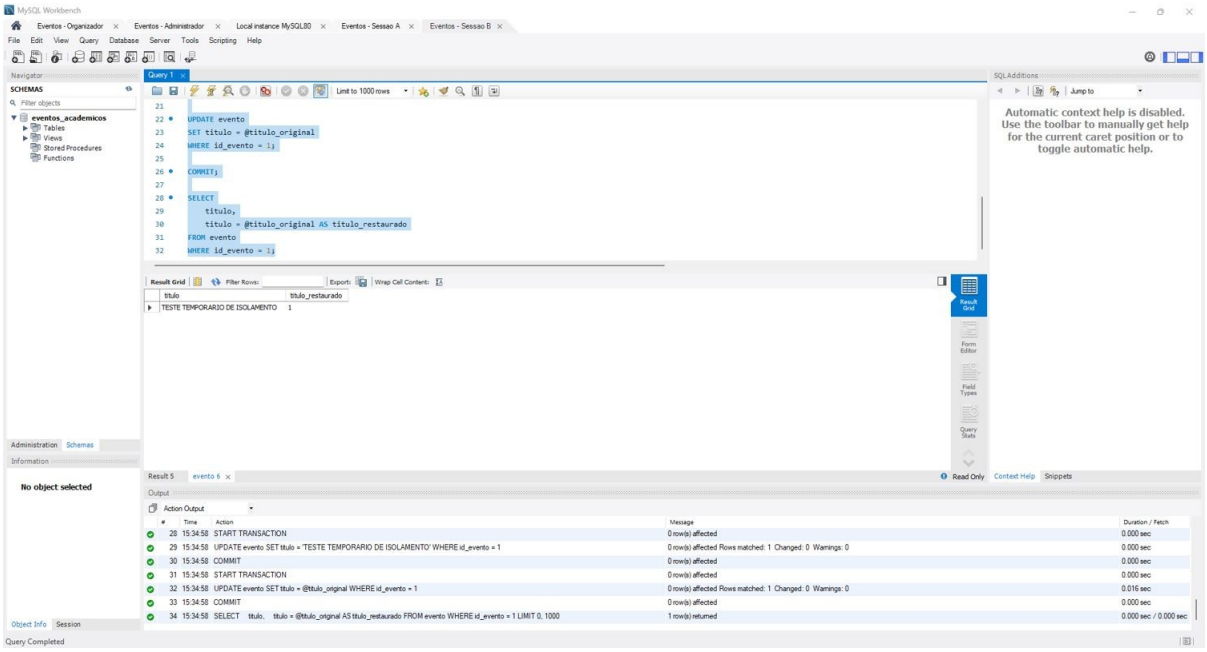


Figura 15 — Dado restaurado após o experimento

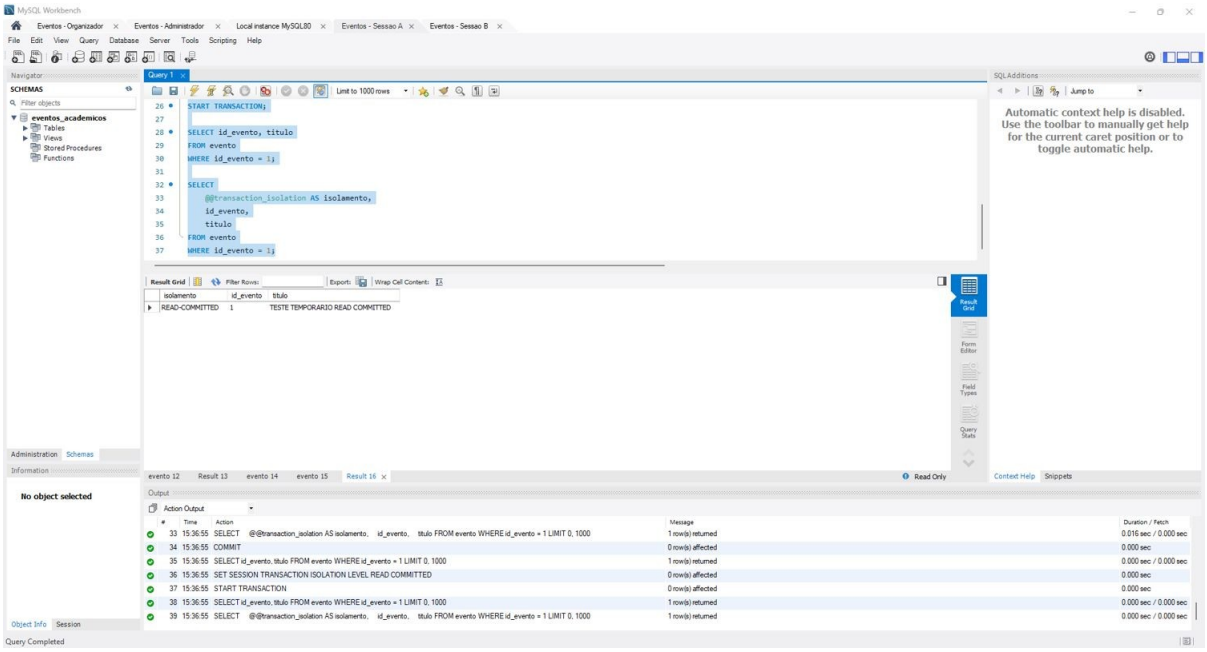


Figura 16 — READ COMMITTED: a segunda leitura enxerga o novo valor (leitura não repetível)

Registros adicionais do experimento (timeout 1205, deadlock 1213 e disputa pela última vaga na aplicação) estão em docs/evidencias/04_concorrencia_sql.txt e 05_sql_injection_e_concorrencia_app.txt.

3.4 Justificativa do nível de isolamento

Adotamos REPEATABLE READ (padrão) combinado com SELECT … FOR UPDATE nos pontos críticos (controle de vagas e encerramento de evento). Os experimentos mostram que READ COMMITTED permitiria que a contagem de confirmadas mudasse entre a leitura e o INSERT; o FOR UPDATE serializa apenas as inscrições da mesma atividade, sem o custo de bloqueio amplo do SERIALIZABLE. O custo assumido é que inscrições simultâneas na mesma atividade esperam em fila e, em ordem inversa de travas, pode ocorrer deadlock (erro 1213); por isso a aplicação trata os erros 1205/1213 com mensagem para tentar novamente.

4. Otimização de consultas (B4)

Foram estudadas três consultas críticas com EXPLAIN ANALYZE antes e depois da criação de um índice (sql/07_estudo_indices.sql), usando carga sintética em volume (sql/08_carga_volume.sql, cerca de 120 mil inscrições, 50 mil pessoas, 6 mil atividades e 6 mil sessões) em um banco separado, eventos_academicos_teste.

#	Consulta crítica	Índice criado	Por que essas colunas, nessa ordem
1	Inscrições por status em um intervalo de datas (BETWEEN + GROUP BY)	idx_inscricao_data (data_inscricao, status_inscricao)	Data primeiro porque o filtro é intervalo (colunas após um intervalo não restringem a busca); status depois só para cobrir o GROUP BY
2	Busca de pessoa por prefixo do nome (LIKE 'Ana S%')	idx_pessoa_nome (nome)	Prefixo fixo permite varredura por faixa no índice
3	Sessões de uma sala em um período, ordenadas	idx_sessao_sala_inicio (id_sala, inicio)	Igualdade primeiro, intervalo depois: o índice entrega as linhas já ordenadas e elimina o Sort

4.1 Resultados

A COMPLETAR: rodar sql/08_carga_volume.sql e depois sql/07_estudo_indices.sql no MySQL do grupo (o print 43 mostrou o banco de teste ainda com 80 pessoas e 264 inscrições, e o EXPLAIN “antes” mostrou rows=0, então ainda não houve medição válida). Preencher a coluna “Grupo” com a mediana de 3 execuções, informar o volume real e anexar os prints de EXPLAIN antes/depois dos 3 estudos.

Estudo	Ref. antes (ms)	Ref. depois (ms)	Grupo antes (ms)	Grupo depois (ms)	Volume (linhas)
1. Inscrições por intervalo de datas	61,1	17,9			120.009
2. Pessoa por prefixo de nome	10,6	2,35			50.009
3. Sessões de uma sala por período	3,55	0,95			6.002

“Ref.” são medições de referência obtidas em MySQL 8.0 com a mesma carga e os mesmos scripts do repositório (registradas em docs/evidencias/07_explain.txt). Elas indicam a ordem de grandeza esperada; os tempos mudam entre máquinas e execuções e devem ser substituídos pelos medidos pelo grupo.

4.2 Leitura dos planos

Estudo 1: antes, o plano era Table scan nas 120 mil linhas com filtro aplicado depois da leitura; depois, Covering index range scan — só a faixa de datas é lida e, como o índice inclui status_inscricao, a tabela nem é consultada. Estudo 2: o LIKE com prefixo fixo passa a usar Index range scan sobre a faixa “Ana S…”. Estudo 3: antes, usava o índice da FK (fk_sessao_sala), lia todas as sessões da sala, filtrava as datas e ordenava (Sort); com (id_sala, inicio) o índice entrega as linhas ordenadas e o Sort desaparece.

4.3 Discussão

(a) Contraexemplo: LIKE '%Souza%' (curinga no início) não usa o índice como busca; o plano varre o índice inteiro e é mais lento que a busca por prefixo. Seria preciso índice FULLTEXT ou outra estratégia.

(b) O ganho do estudo 3 é grande em proporção, mas pequeno em valor absoluto, porque a tabela tem poucas linhas; só importaria em volume muito maior.

(c) Todo índice custa espaço e deixa INSERT/UPDATE um pouco mais lentos; o índice do estudo 1 é o mais caro em espaço por estar na maior tabela.

5. Segurança e controle de acesso (B5)

Papel / conta	Pode	Não pode (comprovado)
r_admin_eventos / usr_admin_eventos	CRUD em todas as tabelas do domínio; gerir usuários; ler auditoria	Apagar auditoria; DROP TABLE (sem DDL)
r_organizador / usr_organizador	Ler o domínio; inserir/atualizar eventos, atividades, inscrições, histórico; inserir certificados	Alterar certificado (REVOKE UPDATE); apagar evento; ler usuario_app e auditoria
r_consulta / usr_consulta	Ler somente vw_agenda_publica e vw_validacao_certificado	Ler tabelas (inclui pessoa) e o relatório
r_autenticacao / usr_app_auth	Ler login, hash, perfil e ativo; atualizar ultimo_login	Ler dados do domínio; alterar perfil

Princípio do menor privilégio: privilégios por tabela e, no perfil de autenticação, por coluna (GRANT e REVOKE em sql/06_seguranca.sql). As contas com senha ficam em arquivo local fora do Git. Cada perfil foi testado com login real em uma conexão própria do Workbench, com um comando permitido e um negado.

5.1 Perfil de consulta (usr_consulta)

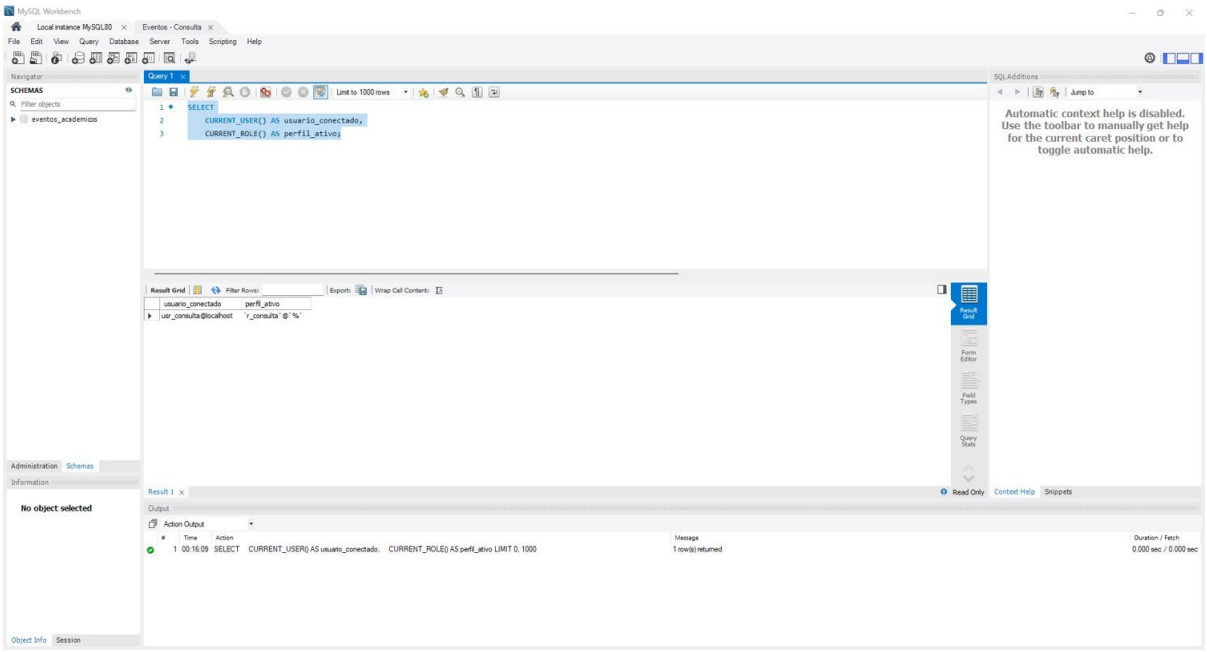


Figura 17 — Conexão com o usuário usr_consulta

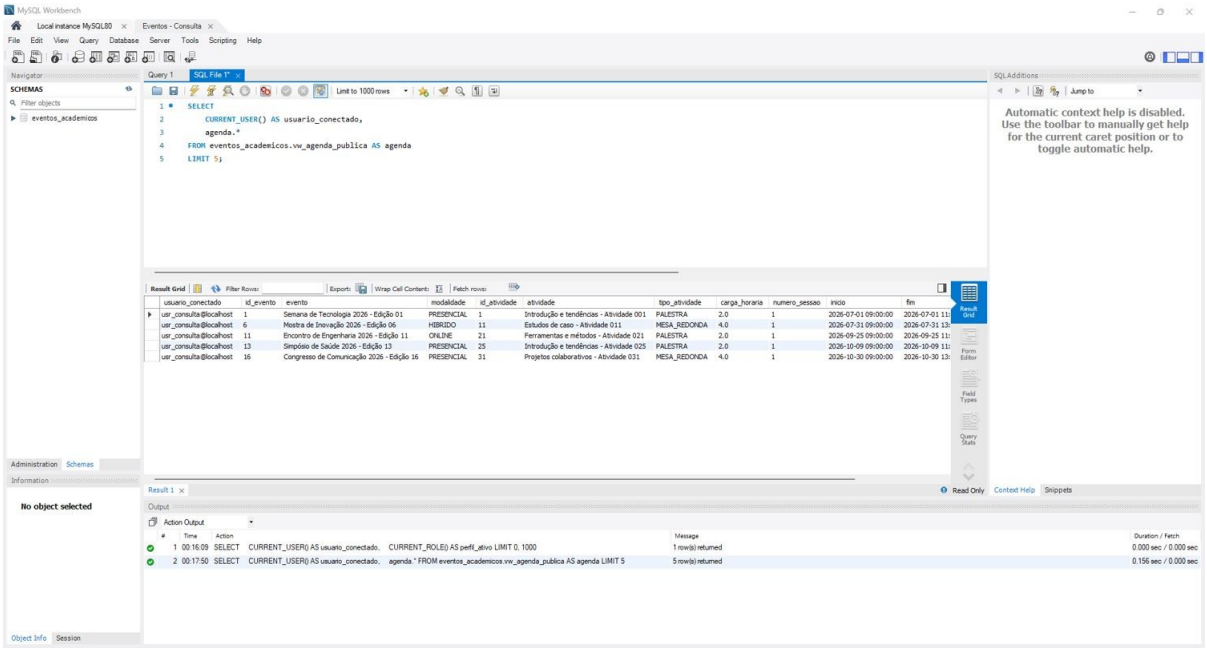


Figura 18 — Consulta à agenda pública — permitida

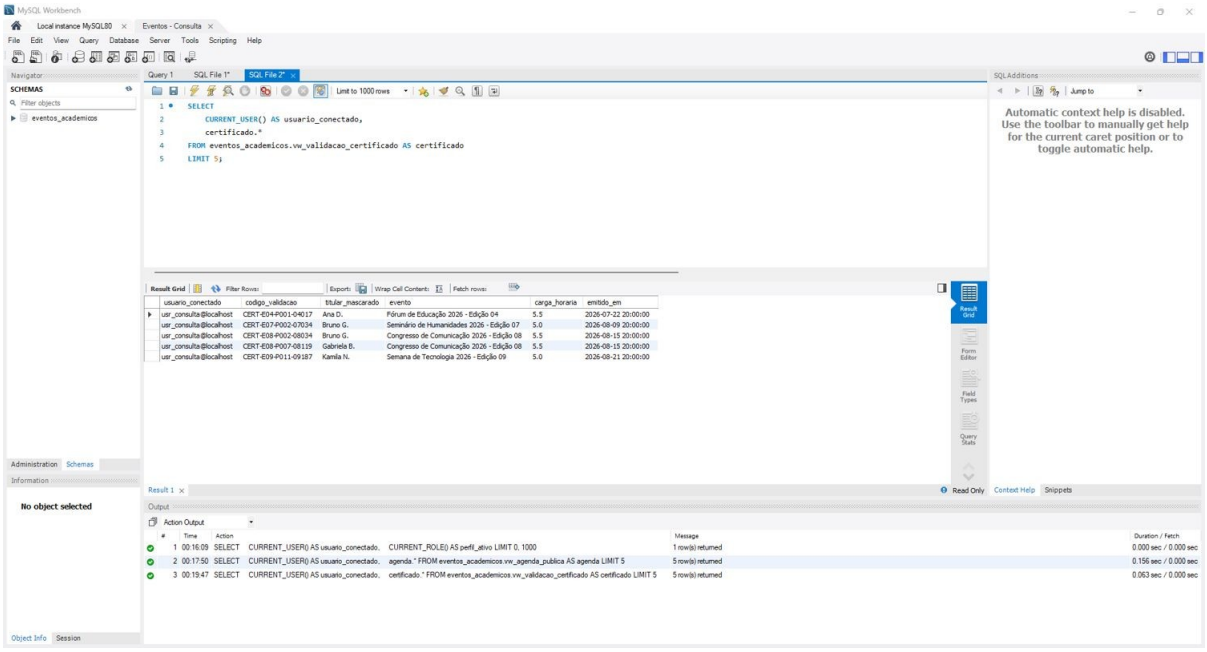


Figura 19 — Consulta à validação de certificados — permitida

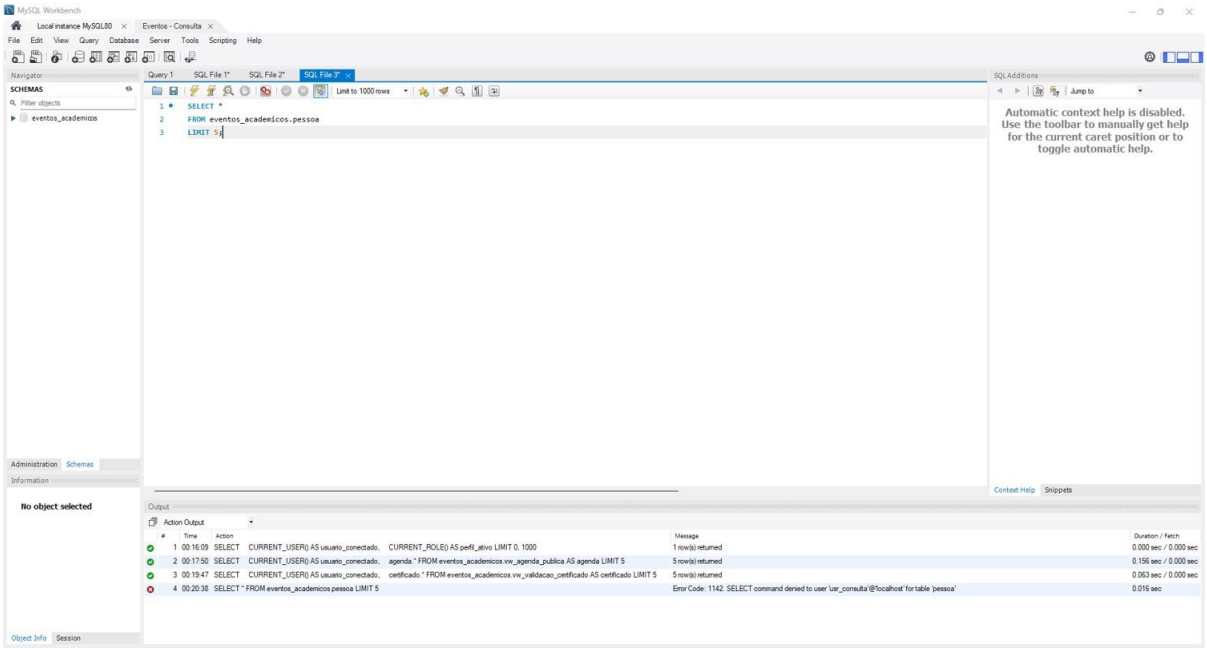


Figura 20 — Consulta a dados pessoais — bloqueada

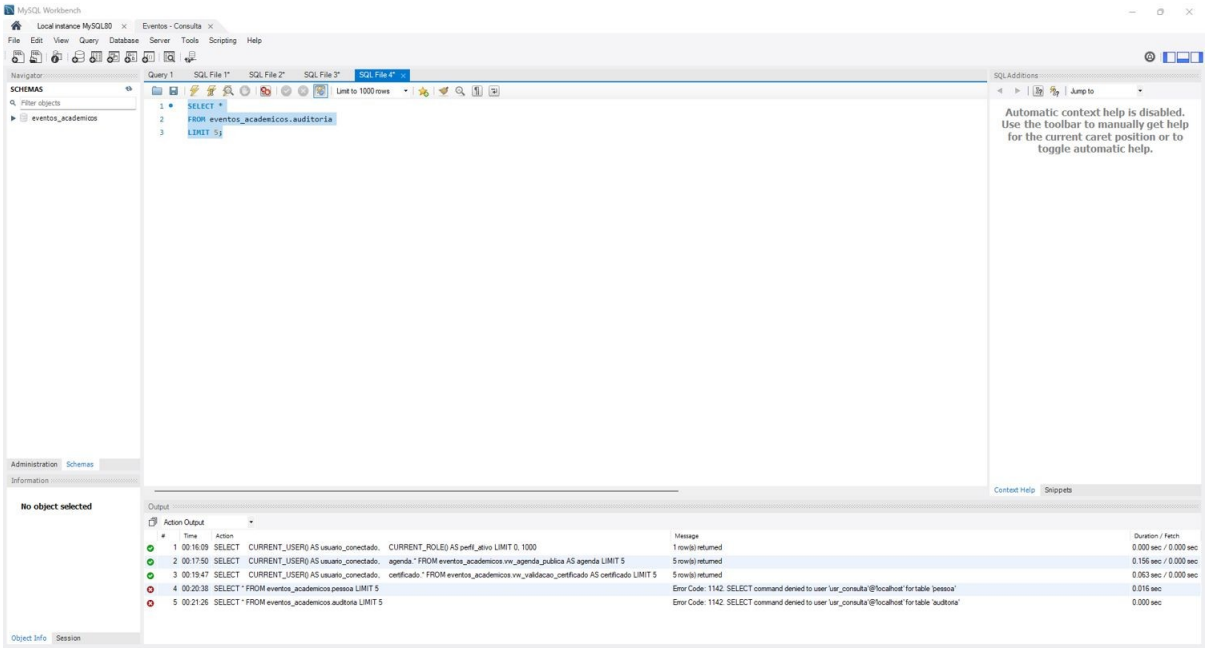


Figura 21 — Consulta à auditoria — bloqueada

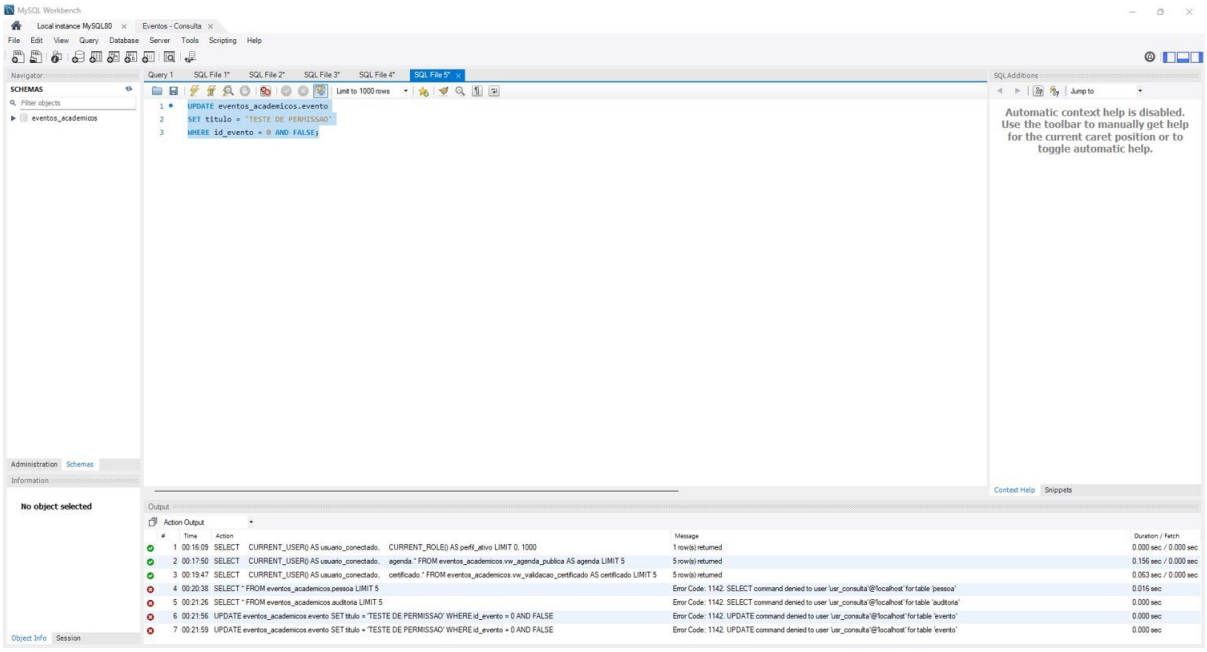


Figura 22 — Alteração de evento — bloqueada

5.2 Perfil de organizador (usr_organizador)

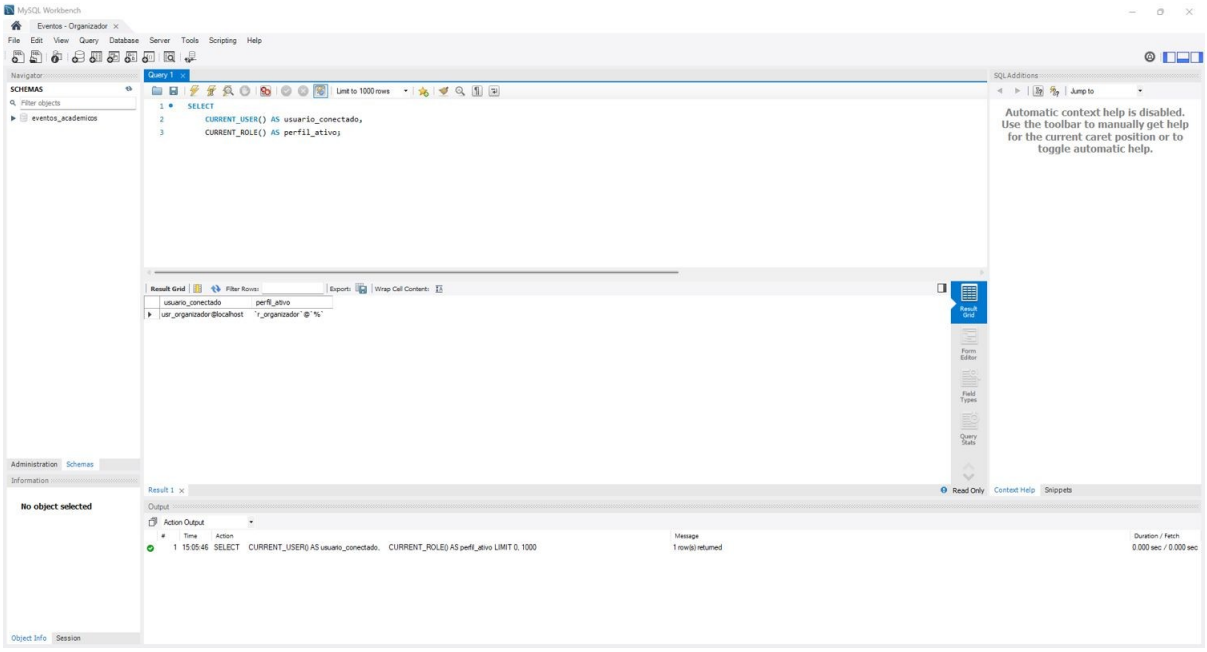


Figura 23 — Conexão com o usuário usr_organizador

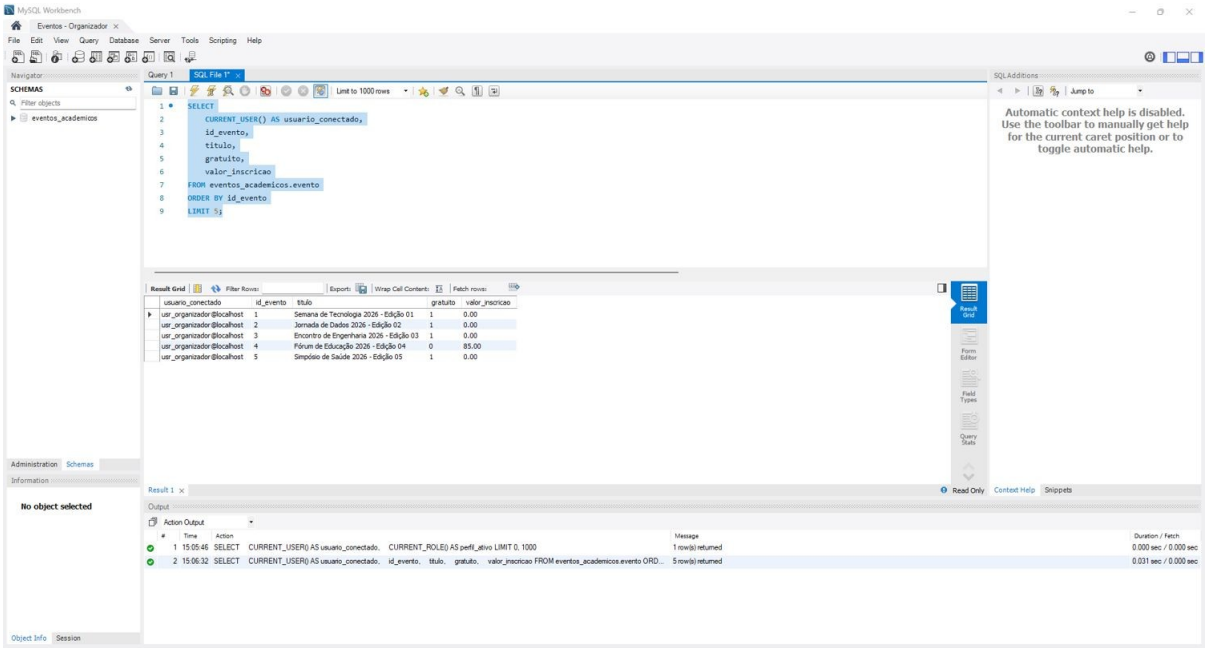


Figura 24 — Consulta de eventos — permitida

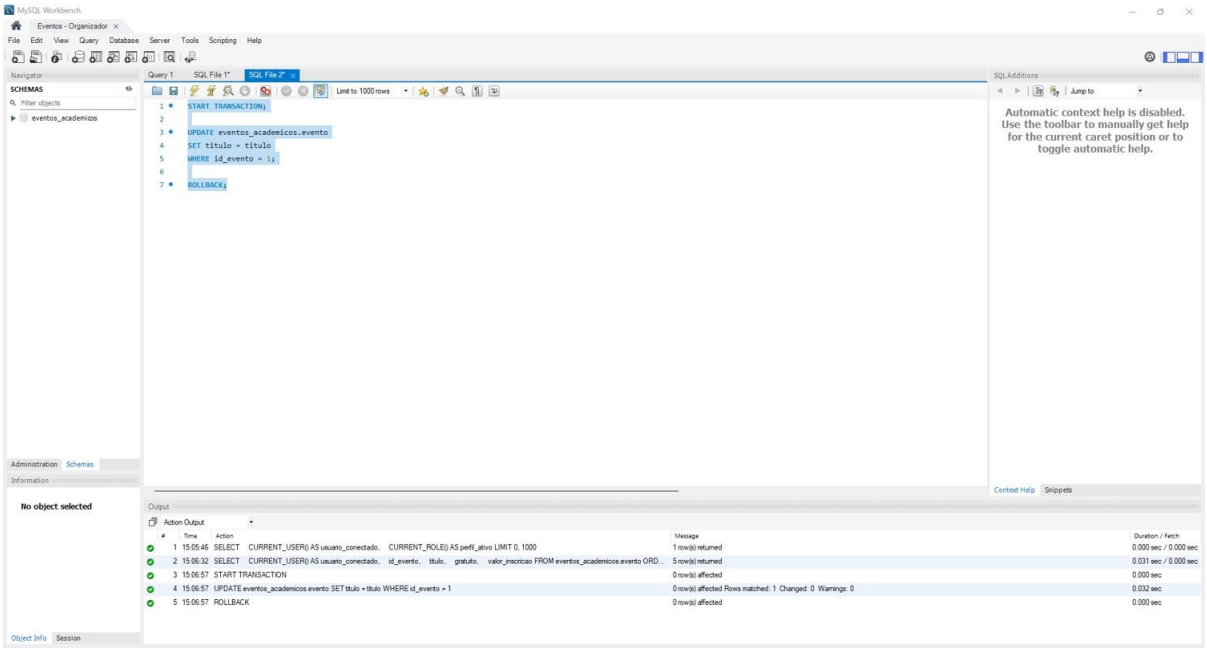


Figura 25 — UPDATE de evento — permitido

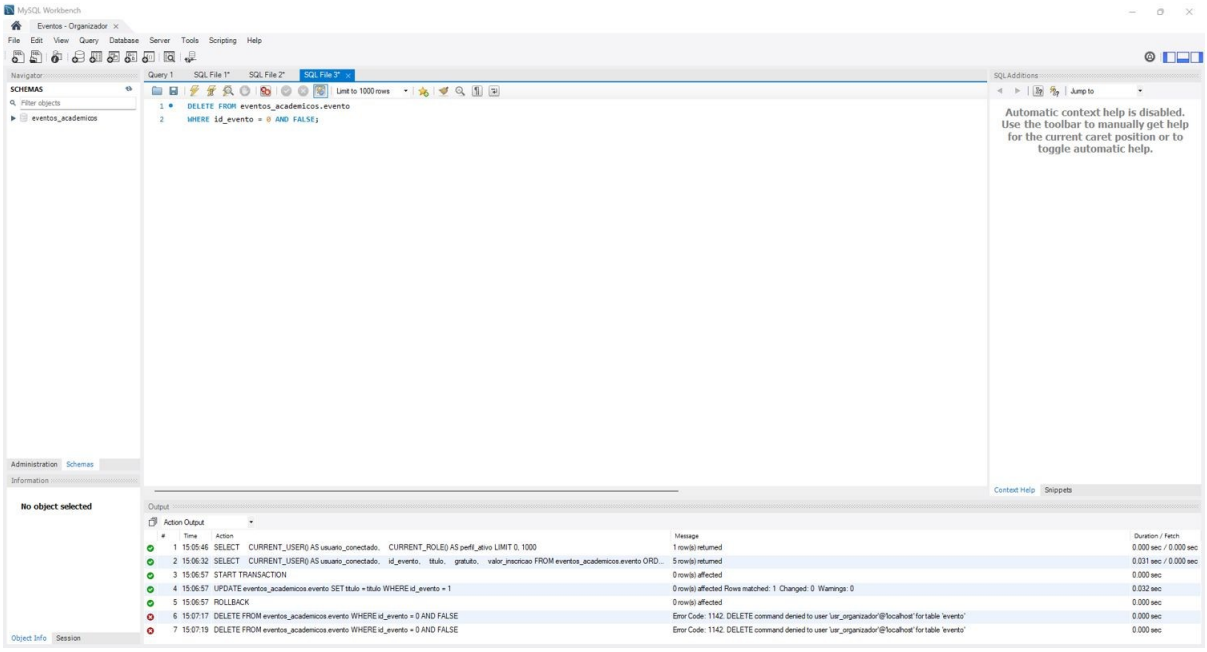


Figura 26 — Exclusão de evento — bloqueada

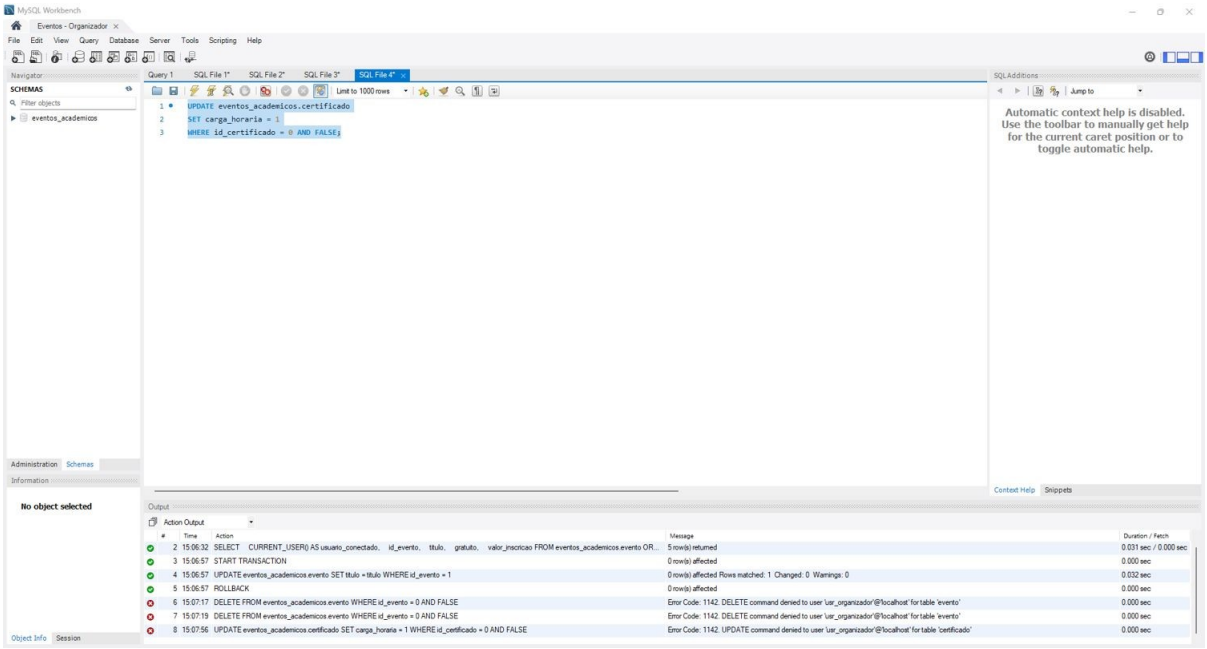


Figura 27 — UPDATE de certificado — bloqueado

5.3 Perfil de administrador (usr_admin_eventos)

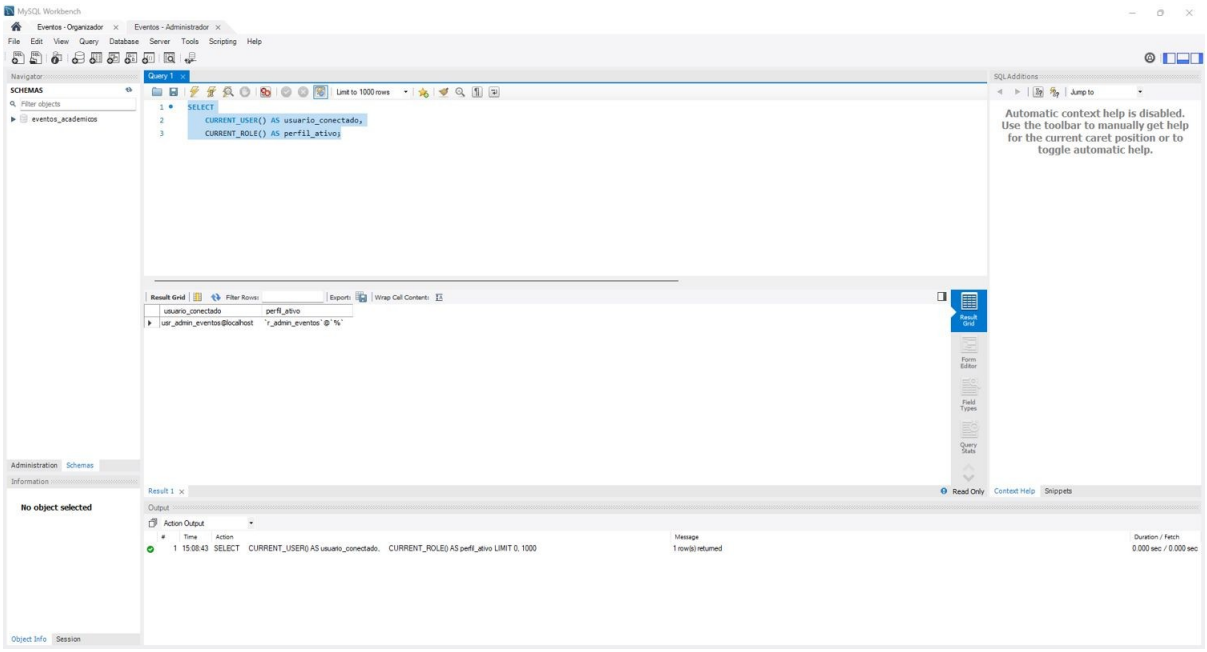


Figura 28 — Conexão com o usuário usr_admin_eventos

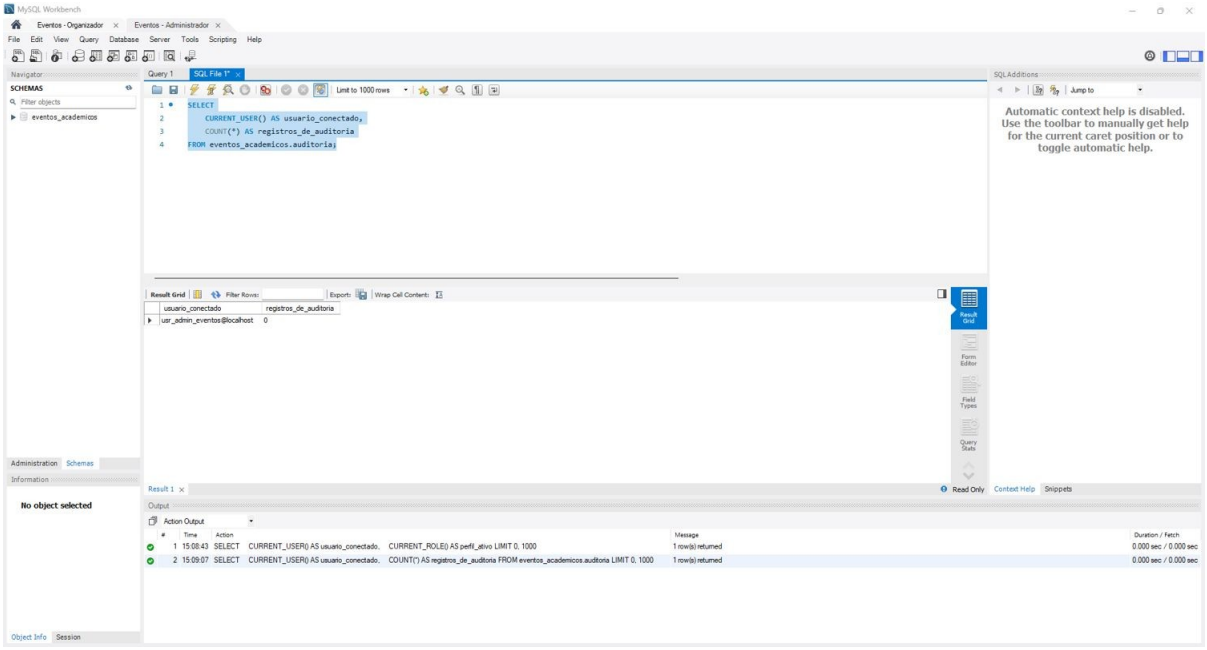


Figura 29 — Leitura da auditoria — permitida

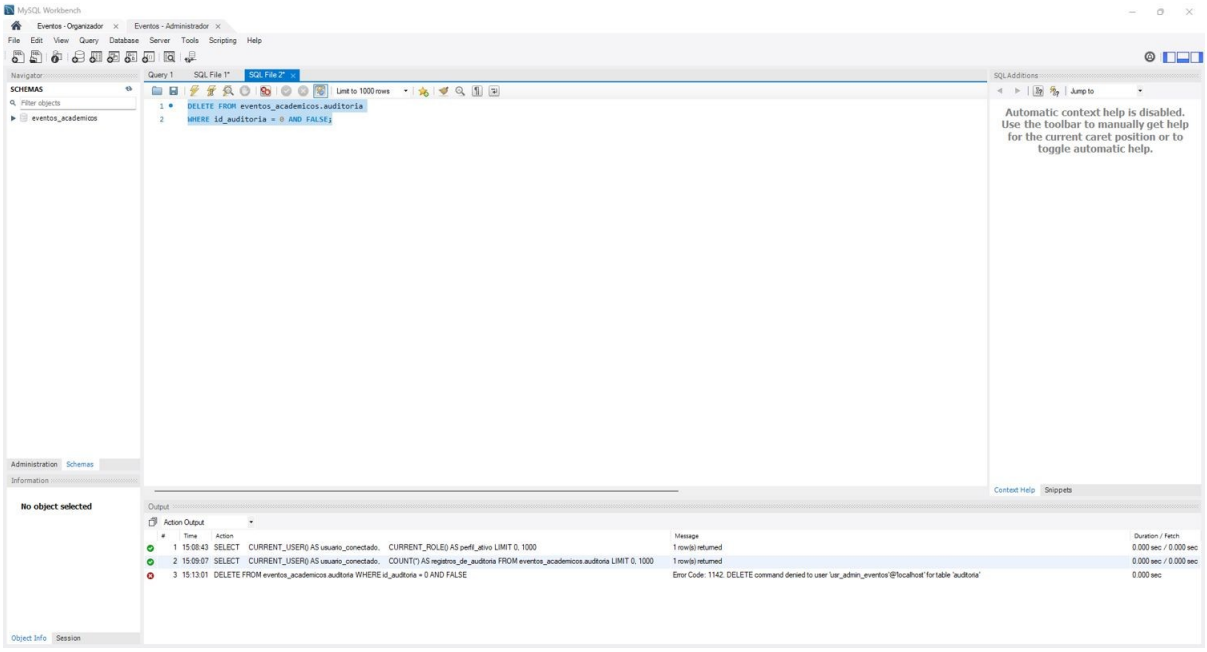


Figura 30 — Exclusão na auditoria — bloqueada

5.4 Visões de restrição

vw_agenda_publica restringe colunas (sem e-mail, telefone, nascimento nem inscritos) e linhas (omite eventos cujo status atual é CANCELADO). vw_validacao_certificado mostra o titular mascarado (por exemplo, “Maria L.”). As visões usam SQL SECURITY DEFINER; por isso o perfil de consulta lê a visão sem acesso às tabelas.

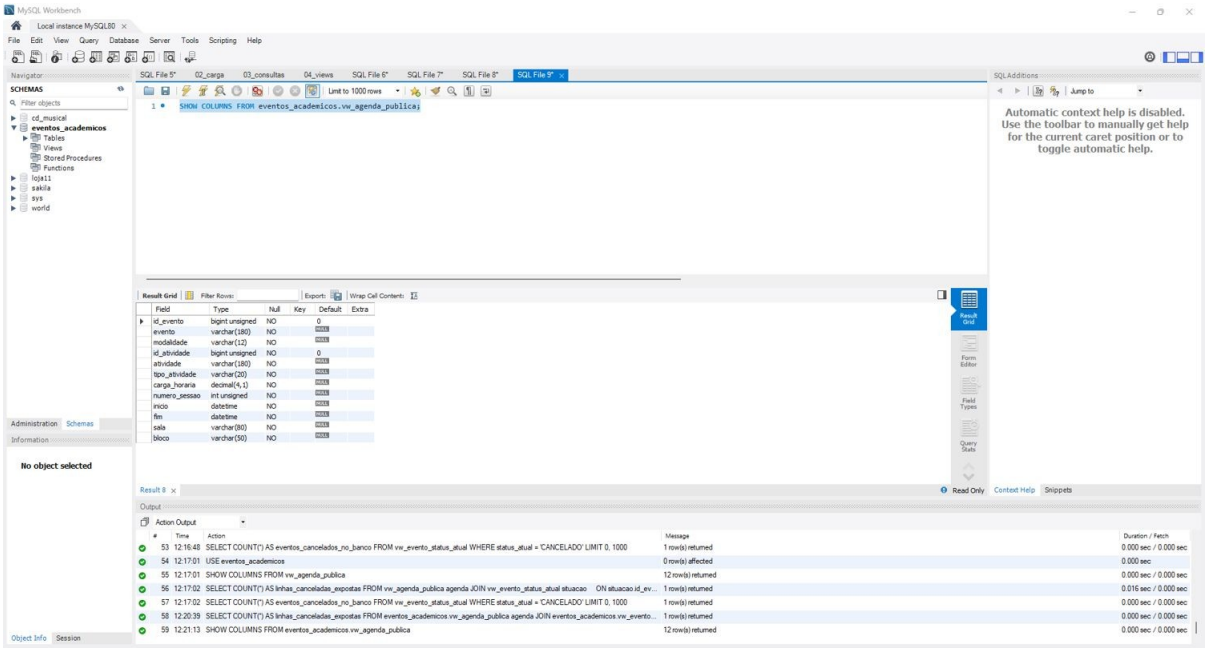


Figura 31 — Colunas da visão vw_agenda_publica

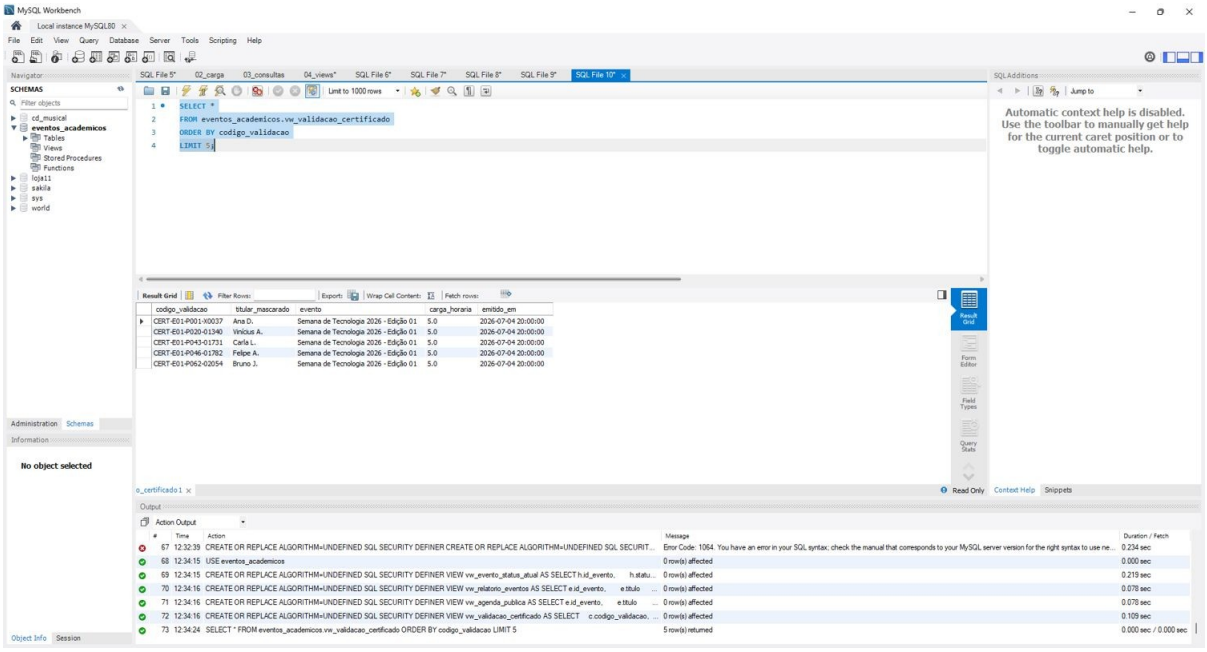


Figura 32 — Visão de certificados com titular abreviado (versão corrigida)

5.5 Senhas

As senhas da aplicação usam bcrypt (custo 12) com sal aleatório embutido no hash ($2b$12$…), gerado pela aplicação. A tabela usuario_app recusa, por CHECK (ck_usuario_app_hash), qualquer valor que não pareça um hash, barrando senha em texto claro por engano.

A COMPLETAR: criar um usuário pela aplicação (python criar_usuario.py admin ADMIN) e anexar print de SELECT login, LEFT(senha_hash, 15) FROM usuario_app; mostrando o prefixo $2b$12$.

5.6 Dados pessoais e cuidados (LGPD)

Dado	Tabela	Cuidado adotado
Nome, e-mail, telefone, data de nascimento	pessoa	Carga 100% fictícia; leitura restrita a admin/organizador; visões públicas não expõem esses campos
Login e senha	usuario_app	Somente hash bcrypt com sal; o perfil de autenticação enxerga apenas as colunas necessárias
Autor das alterações	auditoria	Somente leitura para admin; escrita apenas pelos gatilhos
Nome do titular do certificado	certificado	A validação pública mostra o nome mascarado

O modelo não guarda CPF. Ficam fora do escopo da disciplina, e são registrados como limitações: HTTPS, limite de tentativas de login e criptografia em repouso.

6. Gatilhos e auditoria (B6)

Justificativa pelo domínio. (1) trg_aud_inscricao_atividade_upd audita status e presença da inscrição, que determinam quem tem direito ao certificado; cancelar uma inscrição ou alterar a presença muda esse direito. (2) trg_aud_evento_upd audita datas, modalidade, limite de participantes e valor do evento, informações que afetam os inscritos depois de abertas as inscrições. Alterações em colunas não relevantes (como a descrição) não geram registro.

Autor. A aplicação executa SET @usuario_app = '<login>' ao conectar; o gatilho grava COALESCE(@usuario_app, USER()). Limitação: a variável de sessão identifica o usuário da aplicação, mas não é prova criptográfica.

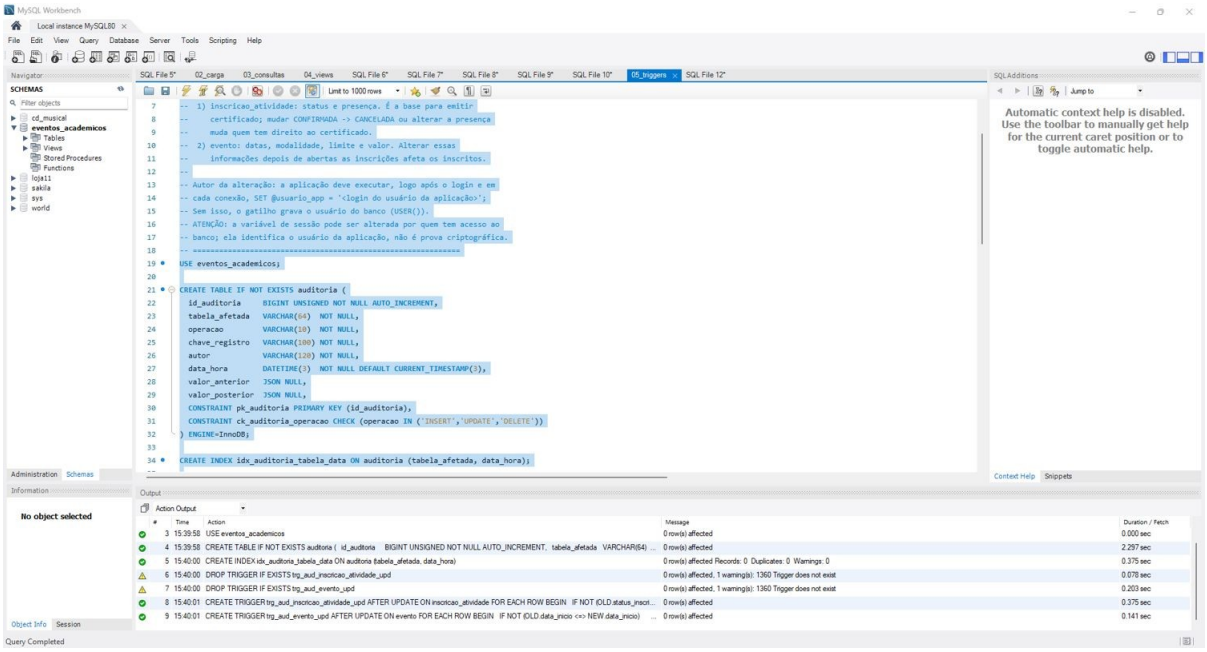


Figura 33 — Criação da tabela auditoria e dos dois gatilhos

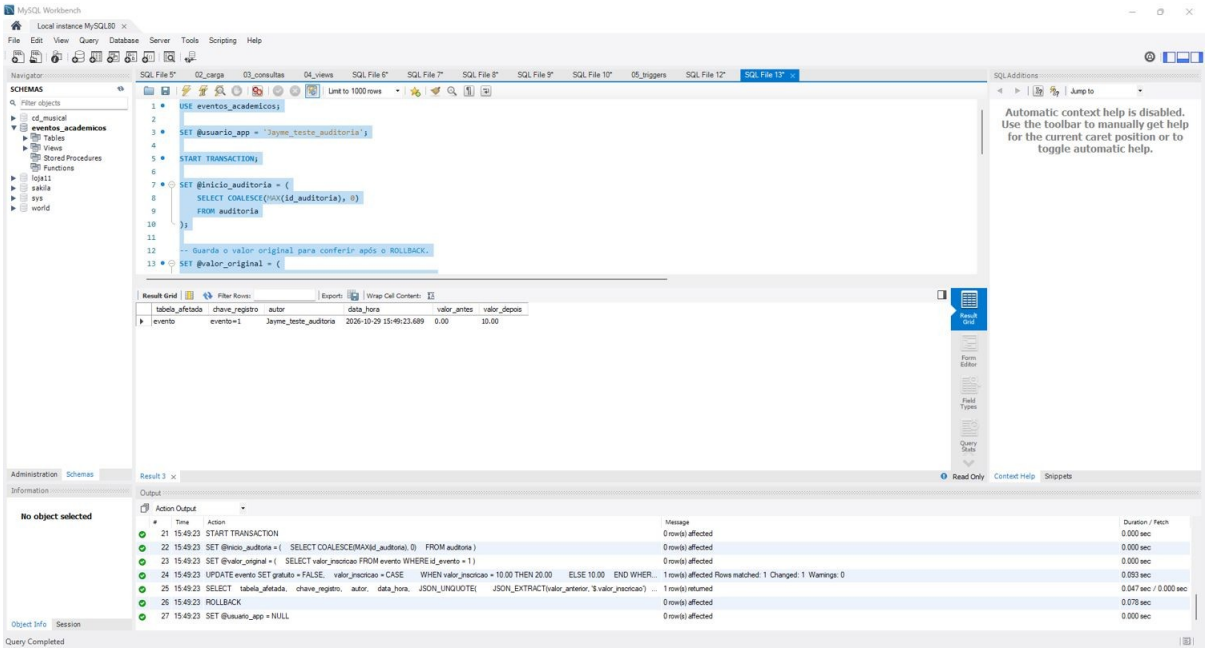


Figura 34 — Auditoria registrada após alteração de evento (autor, data, valores anterior e posterior)

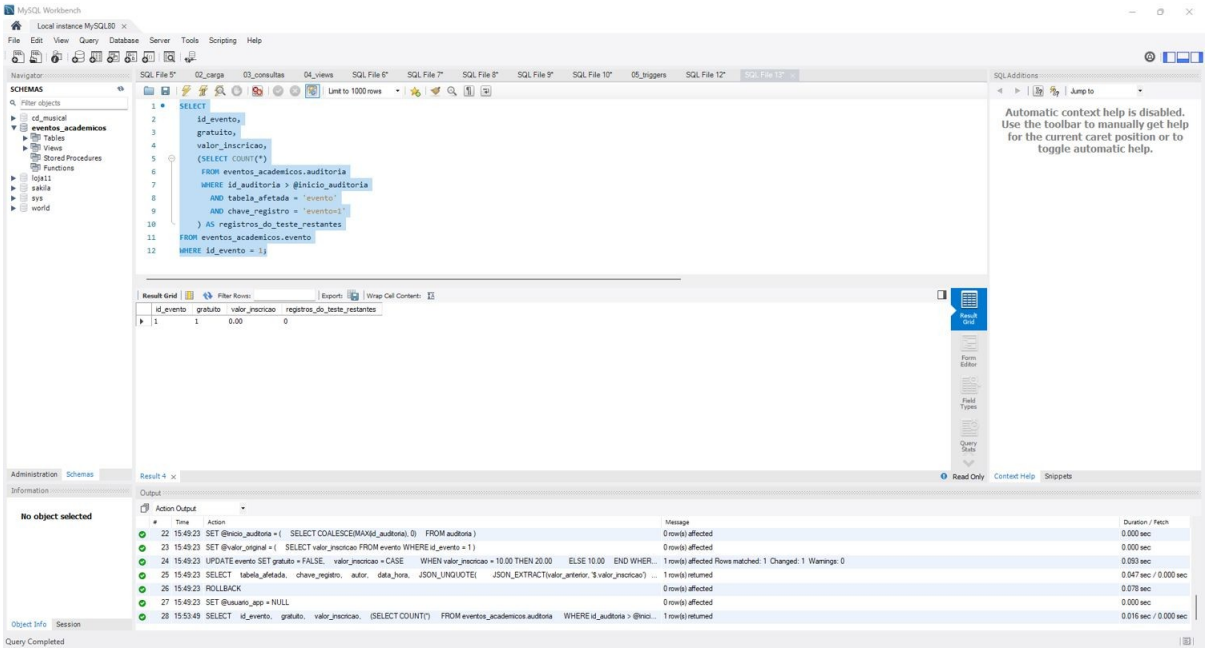


Figura 35 — Evento restaurado após ROLLBACK do teste

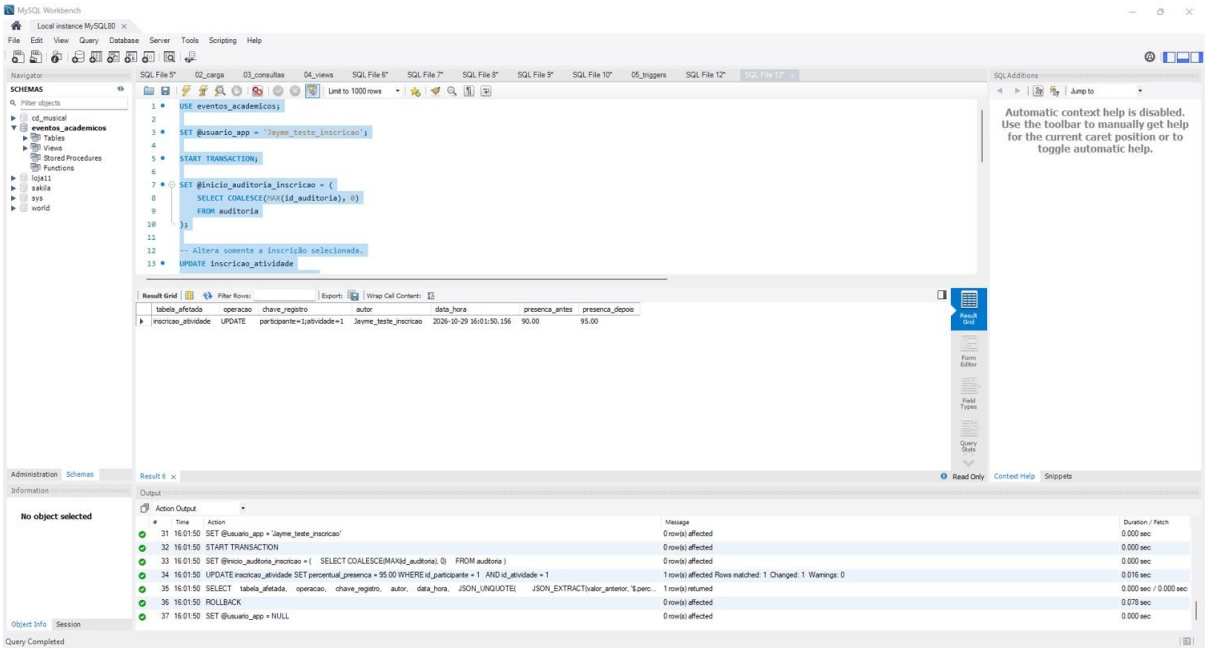


Figura 36 — Auditoria da inscrição: status e presença

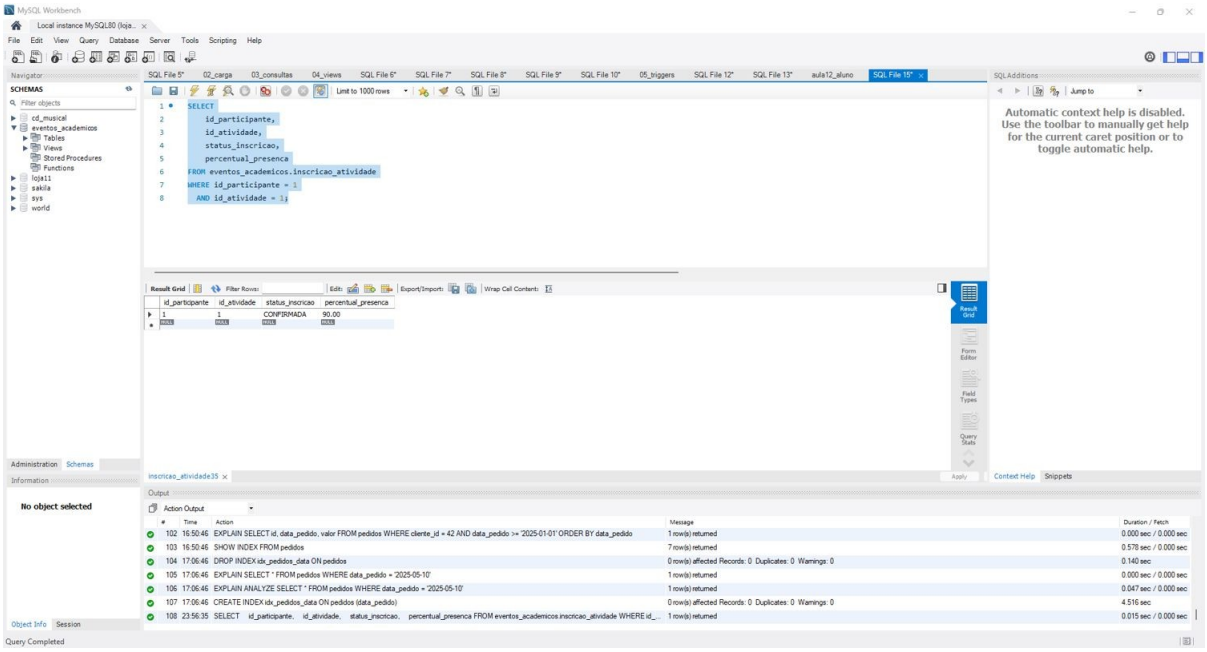


Figura 37 — Inscrição restaurada após ROLLBACK do teste

7. Projeto físico e armazenamento (B7)

7.1 Revisão dos tipos de dados

Coluna / decisão	O que mudaria	Por quê
Chaves BIGINT UNSIGNED (pessoa, evento, atividade…)	Avaliar INT UNSIGNED (até ~4,29 bilhões)	O volume projetado está muito abaixo; BIGINT gasta 4 bytes a mais por linha e em cada índice/FK. Mantido por simplicidade; trocar exigiria recriar as FKs
criado_em TIMESTAMP (pessoa, usuario_app)	Trocar por DATETIME	TIMESTAMP só vai até 2038 e depende do fuso; as demais datas do modelo já são DATETIME
Status/tipos como VARCHAR + CHECK	Manter (não usar ENUM)	ENUM economiza bytes, mas alterar valores exige ALTER TABLE; VARCHAR + CHECK mantém a regra visível no DDL
email VARCHAR(160) com UNIQUE e collation utf8mb4_0900_ai_ci	Manter	O índice único cabe no limite do InnoDB; a collation sem distinção de acento/caixa evita duplicatas por maiúsculas
carga_horaria DECIMAL(4,1) e DECIMAL(5,1)	Manter	O certificado soma atividades, por isso tem um dígito a mais; DECIMAL evita perda de precisão

7.2 Estimativa de crescimento em dois anos

Hipóteses de uso: 120 eventos por ano, 3 atividades por evento, 40 inscrições por atividade, 60 certificados por evento, 2.000 pessoas novas por ano e 0,5 registro de auditoria por inscrição. O tamanho por linha vem da medição em carga sintética (dados + índices).

Tabela	Bytes/linha (medido)	Linhas em 2 anos	Tamanho estimado (MB)
pessoa	227	4.000	0,9
participante	110	4.000	0,4
evento	221	240	0,1
historico_status_evento	106	720	0,1
atividade	292	720	0,2
sessao_atividade	128	1.080	0,1

Tabela	Bytes/linha (medido)	Linhas em 2 anos	Tamanho estimado (MB)
inscricao_atividade	284	28.800	7,8
certificado	—	14.400	3,0
auditoria	—	14.400	4,8
TOTAL			≈ 17,4 (0,02 GB)

As tabelas de maior movimento são inscricao_atividade e auditoria (mais de 95% das linhas). No volume projetado o crescimento é administrável sem particionar.

A COMPLETAR: conferir se as hipóteses de crescimento refletem o uso esperado pelo grupo; se mudarem, recalcular a tabela. Os bytes/linha vêm de docs/evidencias/08_crescimento.txt e podem ser refeitos após a carga em volume.

7.3 Plano de backup

Item	Decisão
O que copiar	Banco eventos_academicos completo (dados, visões, gatilhos, rotinas) com mysqldump --single-transaction --routines --triggers --events. Contas e GRANTs são recriados por 06_seguranca.sql + 06_usuarios.local.sql; .env e backup.cnf ficam fora do Git, em gerenciador de senhas
Frequência	Completo diário às 02:00 (baixo movimento). Se a perda aceitável for menor que 24 h, somar backup incremental por binlog
Retenção	14 dias de diários, 8 semanais e 12 mensais (o script aplica a janela diária; as demais por rotação)
Onde guardar	Regra 3-2-1: 3 cópias, 2 mídias, 1 fora do local
Teste de restauração	restaurar_teste.sh restaura o último dump em um banco temporário e compara a contagem de cada tabela e de gatilhos; executar após mudança de esquema e ao menos uma vez por mês

Teste executado pelo grupo no próprio MySQL:

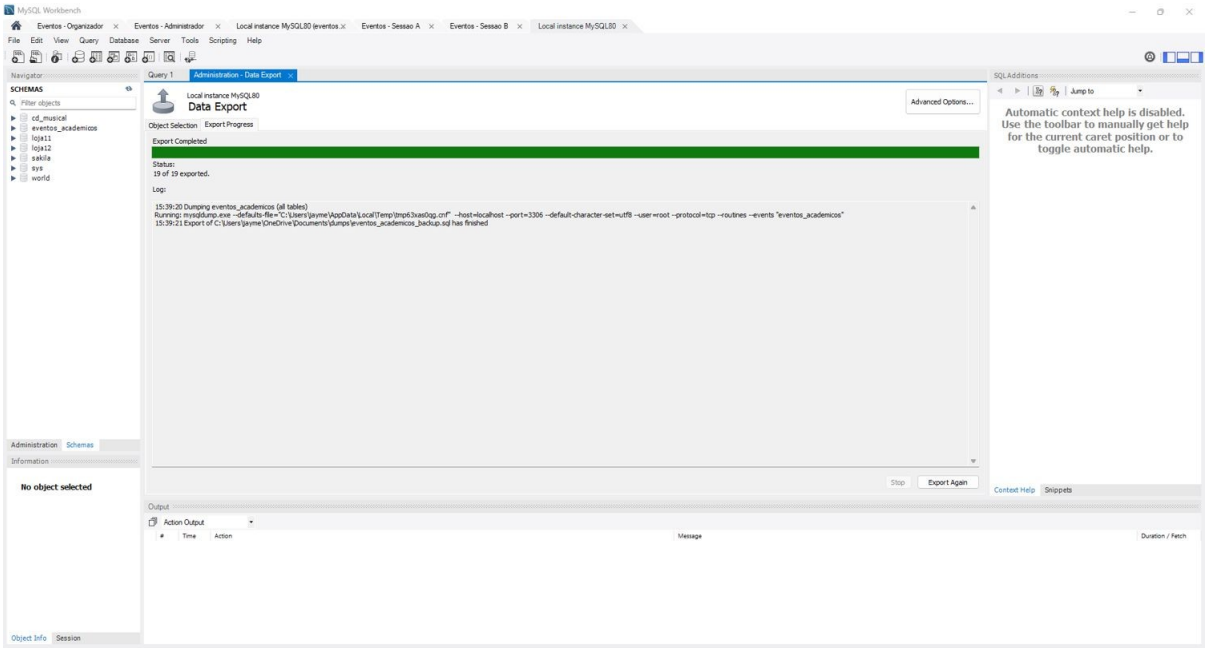


Figura 38 — Exportação do backup concluída

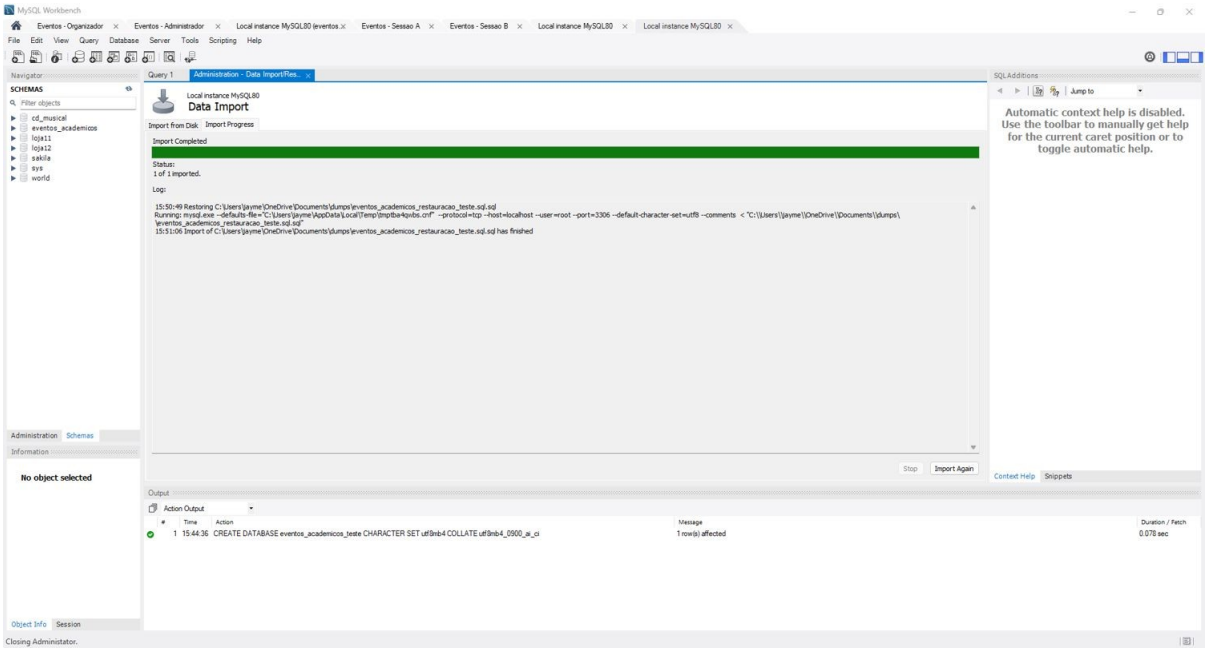


Figura 39 — Restauração em banco temporário concluída

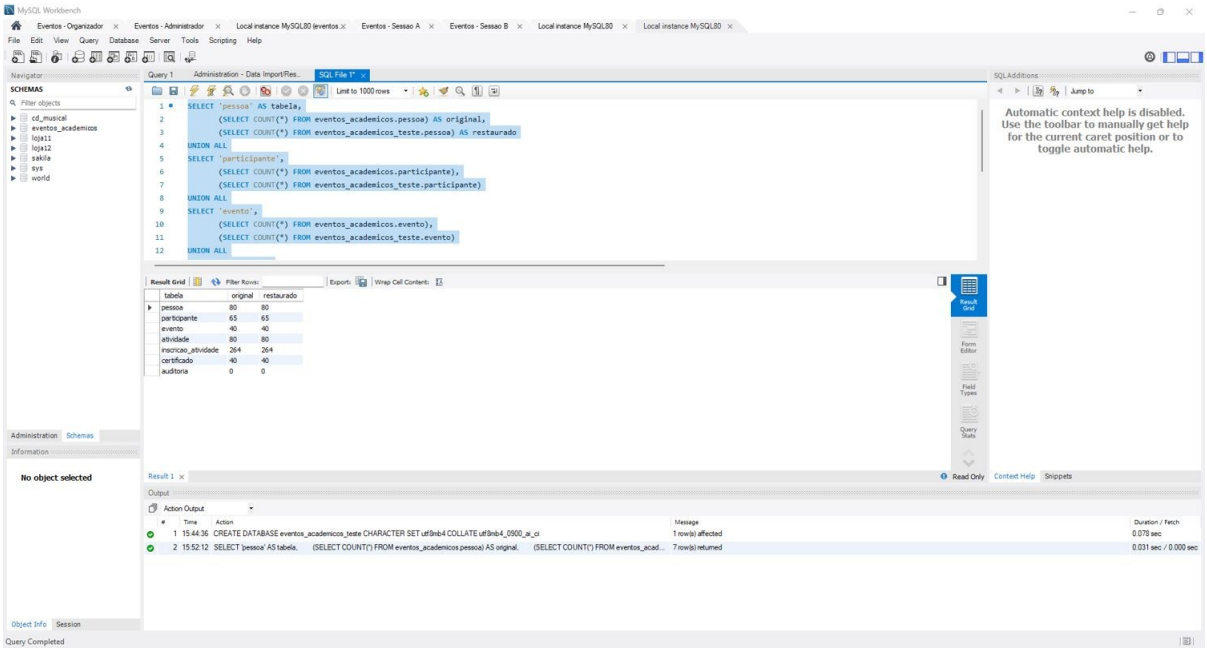


Figura 40 — Comparação das contagens entre original e restaurado

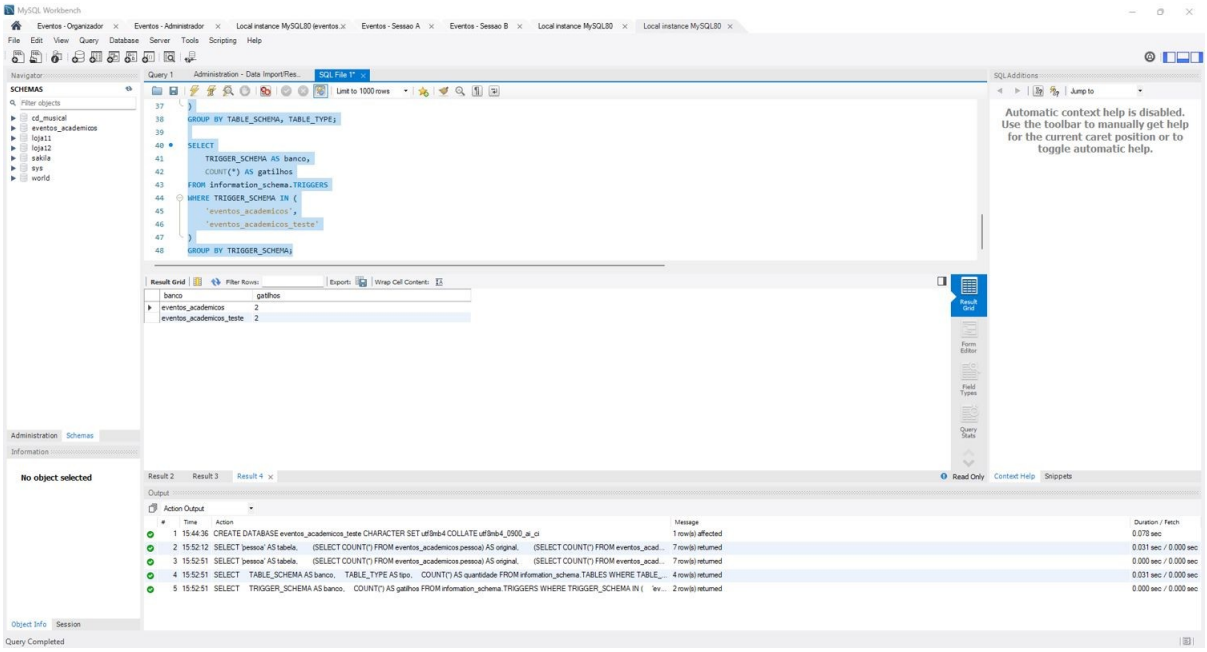


Figura 41 — Objetos (visões, gatilhos) presentes no banco restaurado

7.4 Particionamento e arquivamento

As candidatas pelo volume projetado são inscricao_atividade e auditoria. Restrição técnica: o InnoDB não permite chaves estrangeiras em tabelas particionadas; como inscricao_atividade tem FKs, a proposta para ela é arquivamento (mover inscrições de eventos ENCERRADOS há mais de dois anos para inscricao_atividade_arquivo, preservando os certificados), e não particionamento. A auditoria não tem FK e pode ser particionada por RANGE em data_hora (por ano), exigindo que a PK inclua a coluna de partição, por exemplo PRIMARY KEY (id_auditoria, data_hora). No volume projetado nenhuma das duas é necessária agora; a proposta fica registrada para quando a auditoria passar de milhões de linhas.

8. Declaração de uso de inteligência artificial

Ferramenta. Claude (Anthropic), via claude.ai.

Partes em que foi utilizada. Etapa 1: apoio à estruturação do domínio, revisão de regras de negócio, SQL inicial e documentação. Etapa 2: levantamento do que faltava frente ao enunciado; revisão do 01_ddl.sql; geração inicial dos scripts 04_views, 05_triggers, 06_seguranca, 06_usuarios.exemplo, 07_estudo_indices, 08_carga_volume e 09_transacao_concorrencia; da aplicação Flask (app/) e do teste automático; dos scripts de backup e restauração; e do esqueleto deste relatório. Os arquivos de texto em docs/evidencias/ foram gerados na preparação, com apoio da ferramenta, executando os scripts do repositório.

Como o grupo verificou o resultado. O grupo executou os scripts no próprio MySQL 8.0, pelo Workbench, entre 29 e 30/10/2026, e registrou as capturas de tela usadas neste relatório (pasta docs/evidencias/prints/): carga e consultas; visões; gatilhos e auditoria; os três perfis de acesso, cada um com login real em conexão própria, mostrando um comando permitido e um negado; a transação em duas tabelas com COMMIT, falha provocada e ROLLBACK; o experimento de concorrência com duas sessões em REPEATABLE READ e READ COMMITTED; e o backup com restauração em banco temporário e conferência das contagens. Durante essa execução o grupo encontrou e corrigiu problemas, registrados em docs/evidencias/prints/02_erros_e_correcoes: a abreviação inadequada do titular na visão de certificados, um erro de sintaxe por CREATE duplicado e um teste de restrição que rejeita valor incoerente, com ROLLBACK.

O que o grupo escreveu e decidiu. A escolha do tema (Gestão de eventos acadêmicos, do catálogo do enunciado) e do escopo; a execução e a conferência dos scripts e das evidências em ambiente próprio; e a decisão de adotar REPEATABLE READ com SELECT … FOR UPDATE nos pontos críticos, justificada na seção 3.4.

Qualquer integrante do grupo está apto a explicar as decisões e os scripts deste projeto na arguição.